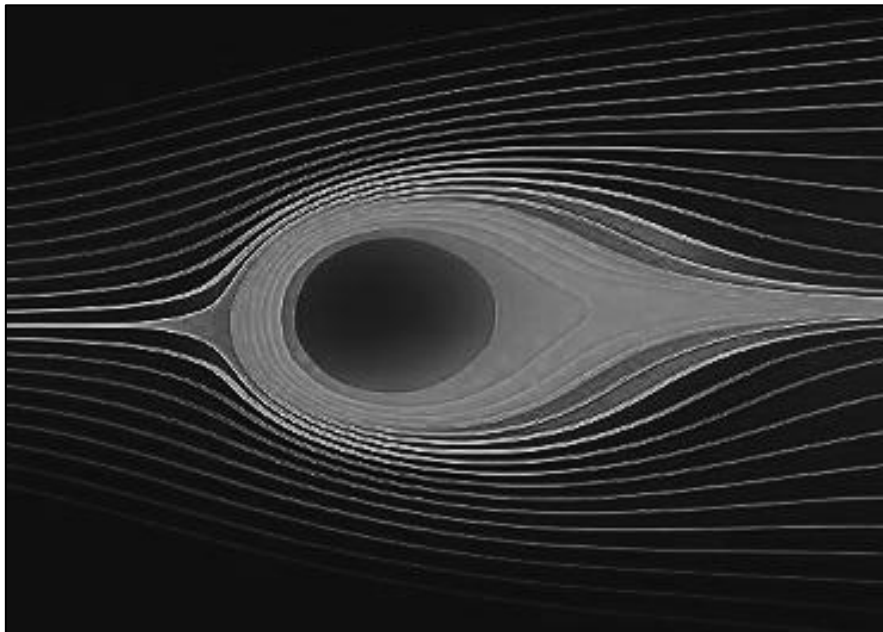


Επίλυση Ασκήσεων Μαθήματος

Υπολογιστική Ρευστομηχανική

2^η Σειρά Ασκήσεων



Ονοματεπώνυμο: Αναστάσιος Μπατατέγας

Αριθμός Μητρώου: 09121431

Εξάμηνο: 9^ο

Σχολή: ΕΜΦΕ

Email: tasosmntdew@gmail.com , ge21431@mail.ntua.gr

Διδάσκων: Ε. Ρετσίνης

Άσκηση 2.1

Υλοποιήστε στην γλώσσα προγραμματισμού Matlab την επίλυση του προβλήματος αρχικών τιμών:

$$y' = 1 + (t - y)^2, \quad y(2) = 1, \quad 2 \leq t \leq 3$$

με τις παρακάτω μεθόδους:

1. Ρητή μέθοδος Euler.
2. Πεπλεγμένη μέθοδος Euler.
3. Runge – Kutta 2^{ης} τάξης.
4. Runge – Kutta 3^{ης} τάξης.
5. Runge – Kutta 4^{ης} τάξης.

Εάν η αναλυτική λύση του παραπάνω προβλήματος είναι η:

$$y(t) = t + \frac{1}{1 - y}, \quad 2 \leq t \leq 3$$

να κάνετε σύγκριση μεταξύ της αναλυτικής και αριθμητικής λύσης υπολογίζοντας το σφάλμα για όλες τις μεθόδους.

Τα αποτελέσματα, εκτός από την αριθμητική τους μορφή, να παρουσιαστούν και σε μορφή διαγραμμάτων.

Λύση

Προς επίλυση του προβλήματος αρχικών τιμών (Π.Α.Τ.):

$$\{y' = f(t, y), \quad f(t, y) \equiv 1 + (t - y)^2, \quad y(2) = 1, \quad 2 \leq t \leq 3\} \quad (2.1)$$

με αναλυτική λύση:

$$y(t) = t + \frac{1}{1 - y}, \quad 2 \leq t \leq 3$$

υλοποιείται, αρχικά, για κάθε ζητούμενη μέθοδο ο αντίστοιχος αλγόριθμος σε γλώσσα Matlab, δημιουργώντας αρχεία συναρτήσεων Matlab (m-function files). Οι υλοποιούμενες συναρτήσεις είναι γενικές, ενώ η επίλυση του συγκεκριμένου Π.Α.Τ. της (2.1) γίνεται σε κατάλληλα αρχεία τύπου m-script. Τα αποτελέσματα για τα οποία παρατίθεται εικόνα από το Command Window της Matlab, εμφανίζονται σε λειτουργία `format compact`, προς ευκρινέστερη απεικόνιση. Η μεταβλητή t αναφέρεται σε πολλά σημεία της παρακάτω ανάλυσης ως «χρόνος», για προφανείς λόγους, ενώ η συνάρτηση $f(t, y)$ του Π.Α.Τ. αναφέρεται ως «συνάρτηση κλίσης», ή και απλά και «κλίση», επίσης για προφανείς λόγους. Για όλες τις παρακάτω περιπτώσεις, επιλέγεται ως «βήμα» της διαμέρισης του πεδίου ορισμού του Π.Α.Τ. (2.1) η τιμή $[h = 0.1]$, η οποία, αν και δεν προσφέρει ιδιαίτερα υψηλή ακρίβεια, είναι κατάλληλη για την ποιοτική σύγκριση και σχολιασμό των μεθόδων, τόσο σε σχέση με την αναλυτική λύση του Π.Α.Τ., όσο και μεταξύ τους.

1. Υλοποιείται σε γλώσσα Matlab ο αλγόριθμος της Ρητής μεθόδου Euler, ακολουθώντας την αλγοριθμική διαδικασία των Σημειώσεων του μαθήματος. Ο πηγαίος κώδικας γράφεται, και αποθηκεύεται, σε ένα αρχείο Matlab συνάρτησης (m-function file) με όνομα “EulerExplicitMethod.m”, και παρατίθεται παρακάτω. Η συνάρτηση δέχεται ως παραμέτρους την συνάρτηση «κλίσης», f , του δεξιού μέλους της εξίσωσης του Π.Α.Τ. (σε τύπο function handle), το χρονικό εύρος, μέσω των t_{initial} και t_{final} , την αρχική συνθήκη y_0 , στο σημείο t_{initial} , καθώς και το μήκος, h , των διαστημάτων της ομοιόμορφης διαμέρισης που πραγματοποιείται στο πεδίο ορισμού του Π.Α.Τ. Σημειώνεται ότι, επειδή η διαμέριση είναι ομοιόμορφη, το μήκος h καθορίζει και τον αριθμό, N , διαστημάτων διαμέρισης, μέσω της σχέσης:

$$N = \left\lfloor \frac{t_{\text{final}} - t_{\text{initial}}}{h} \right\rfloor \equiv \text{floor} \left(\frac{t_{\text{final}} - t_{\text{initial}}}{h} \right) \quad (2.2)$$

όπου ο τελεστής floor, στρογγυλοποιεί το αποτέλεσμα στον πλησιέστερο μικρότερο θετικό ακέραιο (σε περίπτωση που ο N δεν προκύπτει ήδη ακέραιος).

Αρχικά, γίνεται έλεγχος της ορθότητας μορφής των δεδομένων εισόδου. Συγκεκριμένα, ελέγχεται το κατά πόσο το εισαγόμενο function handle, f , περιέχει τον ορθό αριθμό μεταβλητών εντός του, με έναν βρόγχο τύπου while(true). Εντός του βρόγχου δοκιμάζεται (try) η εισαγωγή στο function handle ενός ζεύγους ψευδοτυχαίων τιμών. Η εντολή $f(\text{rand}(1), \text{rand}(1))$ καταλήγει σε σφάλμα (exception) μόνον στην περίπτωση που το function handle δεν περιέχει τον ορθό αριθμό μεταβλητών, και σε αυτή την περίπτωση, το try αγνοείται και στο χωρίο του catch το πρόγραμμα σταματά με την εντολή error(). Επιπλέον, ελέγχεται το κατά πόσο οι εισαγόμενες τιμές για τις παραμέτρους t_{initial} και t_{final} είναι πράγματι αριθμοί (ήτοι πίνακες 1×1).

Μετά τον έλεγχο ορθότητας των δεδομένων εισόδου, η συνάρτηση υλοποιεί τον αλγόριθμο της Ρητής μεθόδου Euler. Προς αυτό τον σκοπό, αρχικοποιείται το διάνυσμα των χρονικών τιμών που αντιστοιχούν στην ομοιόμορφη διαμέριση του διαστήματος ορισμού του (2.1), και εξασφαλίζεται ότι το προκύπτον διάνυσμα είναι μία στήλη (με τον τόνο “ˆ”) στο τέλος της αντίστοιχης εντολής), ενώ ορίζεται και ο αριθμός των διαστημάτων της διαμέρισης, N , βάσει της (2.2). Το διάνυσμα με τις τιμές της επίλυσης του Π.Α.Τ., αρχικοποιείται σε μηδενικές τιμές, και ορίζεται να έχει διάσταση $(N + 1) \times 1$, όπου N ο αριθμός των διαστημάτων διαμέρισης, ενώ επιβάλλεται και η αρχική συνθήκη του Π.Α.Τ. στην τιμή του $y(1)$. Στην συνέχεια, υλοποιείται ο επαναληπτικός βρόγχος for της μεθόδου, όπου για κάθε τιμή της μεταβλητής i από 1 έως και N , οι τιμές της λύσης του Π.Α.Τ. υπολογίζονται βάσει της σχέσης:

$$y_{i+1} = y_i + h \cdot f(t_i, y_i), \quad i = 1, 2, \dots, N, \quad y(1) \equiv y_0 \quad (2.3)$$

η οποία ορίζει και την Ρητή μέθοδο Euler. Τονίζεται ότι, σε αντίθεση με τον αλγόριθμο που παρουσιάζεται στις Σημειώσεις του μαθήματος, όπου οι δείκτες της διαμέρισης λαμβάνουν τιμές στο εύρος ακεραίων $\{0, 1, \dots, N\}$, στην υλοποίηση της μεθόδου, οι δείκτες κυμαίνονται στο διάστημα ακεραίων $\{1, 2, \dots, N + 1\}$, γεγονός που οφείλεται στην σύμβαση αρίθμησης των δεικτών πινάκων της Matlab (το πρώτο στοιχείο μίας δομής πίνακα στην Matlab αριθμείται με το 1). Φυσικά, η «ολική μετατόπιση» των δεικτών κατά μία μονάδα εμπρός δεν μεταβάλλει τίποτε στην αλγοριθμική διαδικασία,

εκτός από το γεγονός ότι η αρχική συνθήκη που αποκαλείται “ y_0 ” αφορά στο στοιχείο του $y(1)$ του πίνακα λύσεως. Ο πλήρης πηγαίος κώδικας που υλοποιεί τα παραπάνω παρουσιάζεται παρακάτω.

MATLAB FUNCTION M-FILE (EulerExplicitMethod.m):

```
function [t,y] = EulerExplicitMethod(f, t_initial, t_final, y0, h)
% Function EulerExplicitMethod.m
%-----
% Euler's Explicit method for I.V.P. with step h:
% {  $y'(t) = f(t,y)$ ,  $y_0 = y(t_i)$ ,  $t_{\text{initial}} \leq t \leq t_{\text{final}}$  }
%
% Input : - function handle f for the slope.
%         - initial time parameter t_initial.
%         - final   time parameter t_final.
%         - initial condition y0.
%         - step size h
%         (no. of partition intervals:  $N = (t_{\text{final}} - t_{\text{initial}})/h$ )
% Output: - vector y = sol. of I.V.P.      ((N+1) x 1).
%         - vector t = partition node values ((N+1) x 1).
%-----

% --- Input validity ---
% Ensure function handle f is properly defined:
while(true)
    try
        f(rand(1), rand(1));
        break;
    catch
        error('Function handle f must take two scalar inputs (t,y).');
    end
end

% Ensure t_initial, t_final are properly defined:
if(length(t_initial) ~= 1)
    error('t_initial must be a number');
elseif(length(t_final) ~= 1)
    error('t_final must be a number');
end

% --- Euler's Explicit method implementation ---
% -----
% --- Initialization ---
% Time domain partition nodes:
t = (t_initial : h : t_final)'; % t_initial=t(1), t_final=t(N+1)
N = floor((t_final-t_initial)/h); % No. of partition intervals N.
% Solution vector initialization:
y = zeros(N+1,1);
y(1) = y0; % Initial condition y(1)=y0.

% --- Main loop ---
for i = 1:N
    y(i+1) = y(i) + h*f(t(i), y(i));
end
end
```

Βάσει της παραπάνω συνάρτησης, η επίλυση του Π.Α.Τ. πραγματοποιείται άμεσα. Στο παρακάτω αρχείο (m-script file), παρουσιάζεται τόσο η επίλυση του δεδομένου Π.Α.Τ.

(2.1), όσο και ο τρόπος υπολογισμού των σφαλμάτων της επίλυσής του με την Ρητή μέθοδο Euler, σε σχέση με την δεδομένη αναλυτική λύση. Τα αποτελέσματα επίλυσης, καθώς και η αντίστοιχη αναλυτική λύση, παρουσιάζονται και γραφικά. Στο m-script αρχείο, μετά τον ορισμό όλων των απαραίτητων παραμέτρων του προς επίλυση Π.Α.Τ. (2.1), ορίζεται ένα διάνυσμα `error_EE`, διαστάσεων $(N + 1) \times 1$, στο οποίο αποθηκεύονται τα απόλυτα σφάλματα μεταξύ των τιμών της αναλυτικής λύσης (η οποία ορίζεται προηγουμένως στο πρόγραμμα ως κατάλληλο function handle, και οι τιμές της οποίας αποθηκεύονται στο διάνυσμα `y_anal`) στους κόμβους της θεωρούμενης διαμέρισης και των τιμών που προκύπτουν από την Ρητή μέθοδο Euler (οι οποίες αποθηκεύονται στο διάνυσμα `y_EE`). Η παρουσίαση των τιμών των διανυσμάτων αυτών στο Command Window γίνεται, για λόγους εύκολης εποπτείας και εύκολης αντιγραφής, μέσω της συνάρτησης `table()` της Matlab. Επιπλέον, οι τιμές των διανυσμάτων στήλης `y_anal` και `y_EE` σχεδιάζονται στο ίδιο διάγραμμα με την εντολή `plot` της Matlab, όπου αξιοποιούνται και διάφοροι «διακόπτες» (switches) για πιο ευπαρουσίαστη γραφική παράσταση (ειδικότερα, η αναλυτική λύση παρουσιάζεται με μία συνεχή γραμμή, χρώματος μπλε, ενώ η λύση από την Ρητή μέθοδο Euler με σημεία, που συμβολίζονται με «x», χρώματος κόκκινο).

MATLAB SCRIPT M-FILE (EulerExplicitMethodEx2_1_Script.m):

```
clear;
clc;

% --- CFD Exercise 2.1 ---
%      --- Part 1 ---
% Solution script (using EulerExplicitMethod.m)
%
% -----
% --- DEFINITION OF PARAMETERS ---
% -----
% --- Definition of function handle ---
f = @(t,y) 1 + (t-y).^2;
% --- Definition of t_initial, t_final ---
t_initial = 2; t_final = 3;
% --- Definition of y0 ---
y0 = 1;
% --- Definition of partition step h ---
h = 0.1;
% --- Definition of accuracy epsilon ---
epsilon = 1e-5;
% --- Definition of number N of intervals ---
N = floor((t_final-t_initial)/h);
% --- Definition of node number vector ---
i = (1:(N+1))';
% --- Definition of time values vector ---
% (this vector will be overwritten below, but it is the same
%      for given h, t_initial and t_final)
t = (t_initial : h : t_final)';
% --- Definition of analytical solution function handle ---
y_analytical = @(t) t + (1-t).^-1;
% --- Definition of analytical solution at partition nodes ---
y_anal = y_analytical(t);
%
% -----
```

```
% --- SOLUTION OF I.V.P. ---
% -----
% --- Euler's Explicit method solution ---
[t,y_EE] = EulerExplicitMethod(f,t_initial,t_final,y0,h);
% --- Error Calculation ---
error_EE = abs(y_EE - y_anal);
% --- Definition of table for data output ---
T = table(i, t, y_anal, y_EE, error_EE);
disp(T);
% --- Plot solution and analytical values vs. t values ---
figure;
plot(t, y_anal, 'b-', 'LineWidth', 0.5); hold on;
plot(t, y_EE, 'rx', 'LineWidth', 0.5);
grid on;
xlabel('Time (t)');
ylabel('Solution y(t)');
title('Euler's Explicit method (h = 0.1)');
legend('Analytical Solution', 'Euler's Explicit Method', 'Location',
'Best');
% End of script.
```

i	t	y_anal	y_EE	error_EE
1	2	1	1	0
2	2.1	1.1909	1.2	0.0090909
3	2.2	1.3667	1.381	0.014333
4	2.3	1.5308	1.5481	0.017307
5	2.4	1.6857	1.7046	0.018901
6	2.5	1.8333	1.853	0.019638
7	2.6	1.975	1.9948	0.019836
8	2.7	2.1118	2.1315	0.019693
9	2.8	2.2444	2.2638	0.019338
10	2.9	2.3737	2.3925	0.018851
11	3	2.5	2.5183	0.018287

fx >> |

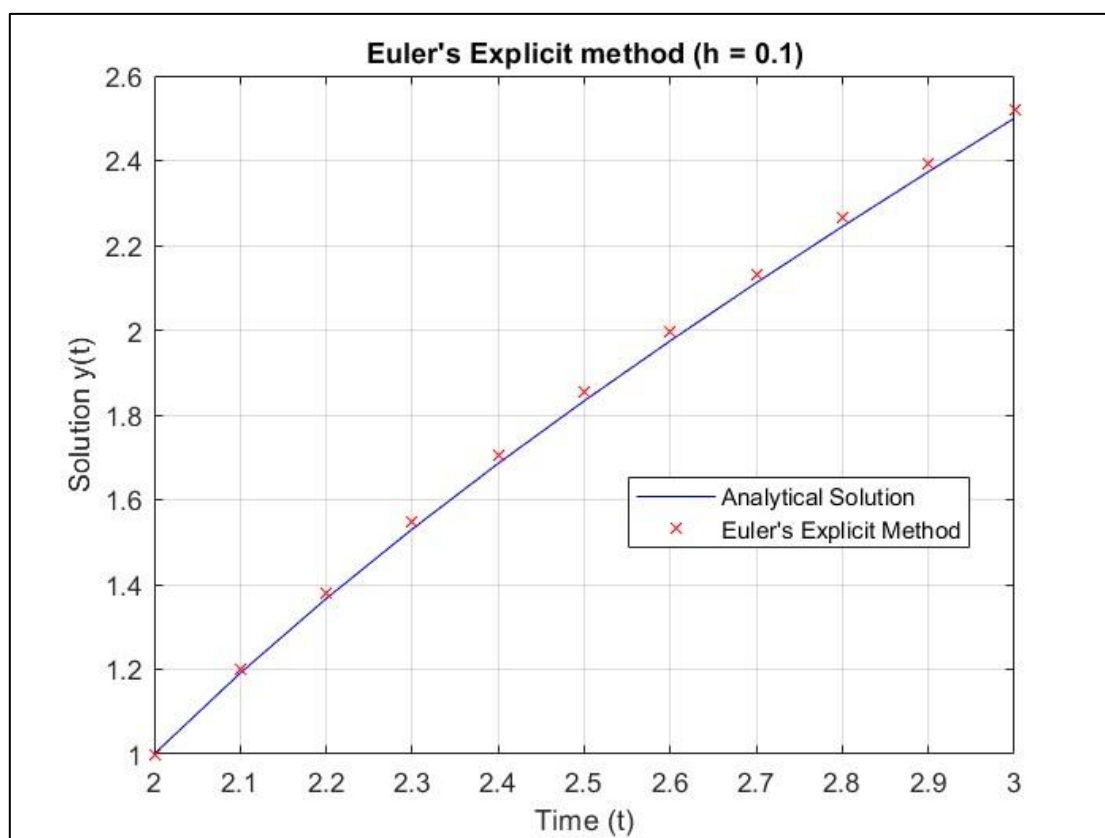
Τρέχοντας το παραπάνω m-script αρχείο, λαμβάνουμε στο τερματικό του Matlab την λύση του δεδομένου Π.Α.Τ. με την Ρητή μέθοδο Euler, όπως φαίνεται παραπάνω (σε μορφή εκτύπωσης που καθορίζεται από την συνάρτηση `table`). Ως «λύση» του Π.Α.Τ. εννοείται η φυσικά ο προσδιορισμός των τιμών της συνάρτησης που επιλύει το Π.Α.Τ. στους κόμβους της διαμέρισης. Λαμβάνουμε επίσης την γραφική παράσταση της αναλυτικής λύσης, καθώς και τον αριθμητικά υπολογιζόμενων με την Ρητή μέθοδο Euler σημείων, στους κόμβους της διαμέρισης (Βλ. Διάγραμμα Ι παρακάτω).

Για καλύτερη εποπτεία και μετέπειτα ανάλυση και σχολιασμό, τα αποτελέσματα αντιγράφονται στον Πίνακα Ι που παρατίθεται παρακάτω, πριν το Διάγραμμα Ι. Στα

παρακάτω εδάφια, τα αποτελέσματα θα εμφανίζονται *μόνον* σε μορφή πινάκων, και θα παραλειφθεί η επίδειξη του παραγόμενου αποτελέσματος στο Command Window του Matlab, για λόγους σμίκρυνσης του μήκους του κειμένου, καθώς και για διαυγέστερη εποπτεία των παραγόμενων αποτελεσμάτων.

Επίλυση Π.Α.Τ. – Ρητή Μέθοδος Euler				
i	Τιμές t	Αναλυτική Λύση $y_{\text{analytical}}$	Ρητή Μέθοδος Euler y_{EE}	Σφάλμα Ρητής Μ. Euler error_{EE}
1	2.0	1.0000	1.0000	0.00000
2	2.1	1.1909	1.2000	0.0090909
3	2.2	1.3667	1.3810	0.014333
4	2.3	1.5308	1.5481	0.017307
5	2.4	1.6857	1.7046	0.018901
6	2.5	1.8333	1.8530	0.019638
7	2.6	1.9750	1.9948	0.019836
8	2.7	2.1118	2.1315	0.019693
9	2.8	2.2444	2.2638	0.019338
10	2.9	2.3737	2.3925	0.018851
11	3.0	2.5000	2.5183	0.018287

Πίνακας Ι.



Διάγραμμα Ι.

Σημειώνεται ότι τα δεδομένα του Πίνακα Ι αναγράφονται με 4 δεκαδικά ψηφία, όπως εμφανίζονται και στο Command Window, υπό την εντολή `format default`, παρόλο που η Matlab αποθηκεύει τις τιμές των αποτελεσμάτων σε περισσότερα δεκαδικά

ψηφία (16 σημαντικά ψηφία). Ωστόσο, η παρεχόμενη ακρίβεια της Ρητής μεθόδου Euler (καθολικό σφάλμα $O(h)$ – μέθοδος πρώτης τάξης), για την επιλεγόμενη τιμή βήματος $h = 0.1$ αποδίδει *προσεγγιστικά* ακρίβεια στο πρώτο δεκαδικό ψηφίο, γεγονός που φαίνεται άμεσα και από τις τιμές του Πίνακα I, καθώς η λύση μέσω της μεθόδου ταυτίζεται με την αναλυτική λύση *το πολύ* στο πρώτο δεκαδικό ψηφίο. Επομένως, η «αποκοπή» των δεκαδικών ψηφίων (η οποία δεν έχει εφαρμοστεί στα σημεία του Διαγράμματος I), εκτός του ότι πραγματοποιείται για λόγους ευκρίνειας των αποτελεσμάτων, είναι και δικαιολογημένη λόγω της παρεχόμενης ακρίβειας της λύσης από τον αλγόριθμο.

2. Η επίλυση του συστήματος μέσω της Πεπλεγμένης μεθόδου Euler (μέθοδος Heun) ακολουθεί όμοια βήματα με τα παραπάνω, με την μόνη διαφορά ότι, ως *επαναληπτική μέθοδος* (iterative method), η υλοποίησή της είναι ελαφρώς διαφορετική – τόσο δομικά όσο και αλγοριθμικά. Υλοποιείται σε γλώσσα Matlab ο αλγόριθμος της Πεπλεγμένης μεθόδου Euler, ακολουθώντας την αλγοριθμική διαδικασία των Σημειώσεων του μαθήματος. Ο πηγαίος κώδικας γράφεται, και αποθηκεύεται, σε ένα αρχείο Matlab συνάρτησης (m-function file) με όνομα “EulerImplicitMethod.m”, και παρατίθεται παρακάτω. Η συνάρτηση δέχεται ως παραμέτρους την συνάρτηση «κλίσης», f , του δεξιού μέλους της εξίσωσης του Π.Α.Τ. (σε τύπο function handle), το χρονικό εύρος, μέσω των $t_{initial}$ και t_{final} , την αρχική συνθήκη y_0 , στο σημείο $t_{initial}$, το μήκος, h , των διαστημάτων της ομοιόμορφης διαμέρισης που πραγματοποιείται στο πεδίο ορισμού του Π.Α.Τ., αλλά και μία επιλεγόμενη ακρίβεια ϵ (η οποία αφορά στο *επαναληπτικό* τμήμα της μεθόδου).

Αρχικά, γίνεται έλεγχος της ορθότητας μορφής των δεδομένων εισόδου, ο οποίος είναι πανομοιότυπος με τον έλεγχο στην υλοποίηση της Ρητής μεθόδου Euler. Η μόνη προσθήκη έγκειται στον έλεγχο του κατά πόσο η εισαγόμενη ακρίβεια ϵ είναι θετική, μέσω κατάλληλης συνθήκης `if`.

Μετά τον έλεγχο ορθότητας των δεδομένων εισόδου, η συνάρτηση υλοποιεί τον αλγόριθμο της Πεπλεγμένης μεθόδου Euler. Προς αυτό τον σκοπό, αρχικοποιείται το διάνυσμα των χρονικών τιμών που αντιστοιχούν στην ομοιόμορφη διαμέριση του διαστήματος ορισμού του (2.1), και εξασφαλίζεται ότι το προκύπτον διάνυσμα είναι μία στήλη (με τον τόνο “’” στο τέλος της αντίστοιχης εντολής), ενώ ορίζεται και ο αριθμός των διαστημάτων της διαμέρισης, N , βάσει της (2.2). Το διάνυσμα με τις τιμές της επίλυσης του Π.Α.Τ., αρχικοποιείται σε μηδενικές τιμές, και ορίζεται να έχει διάσταση $(N + 1) \times 1$, όπου N ο αριθμός των διαστημάτων διαμέρισης, ενώ επιβάλλεται και η αρχική συνθήκη του Π.Α.Τ. στην τιμή του $y(1)$. Στην συνέχεια, υλοποιείται ο επαναληπτικός βρόγχος της μεθόδου, όπου για κάθε τιμή της μεταβλητής i από 1 έως και N , οι τιμές της λύσης του Π.Α.Τ. υπολογίζονται *επαναληπτικά*, βάσει της σχέσης:

$$y_{i+1}^{(k+1)} = y_i + \frac{h}{2} \cdot \left[f(t_i, y_i) + f(t_i, y_{i+1}^{(k)}) \right], \quad i = 1, 2, \dots, N, \quad y_1 \equiv y(1) \equiv y_0 \quad (2.4)$$

η οποία ορίζει και την Πεπλεγμένη μέθοδο Euler. Η επαναληπτική μέθοδος αξιοποιεί την μέθοδο Newton – Raphson, και ως *συνθήκη τερματισμού* την «σταθμισμένη» νόρμα μεγίστου (Βλ. Άσκηση 1.3, στην επίλυση της 1^{ης} Σειράς Ασκήσεων):

$$\frac{\|y_{i+1}^{(k+1)} - y_{i+1}^{(k)}\|_{\infty}}{\|y_{i+1}^{(k+1)}\|_{\infty}} < \varepsilon \quad (2.5)$$

όπου η παράμετρος ακρίβειας $\varepsilon \equiv \text{epsilon}$, εισάγεται από τον χρήστη (και θα ληφθεί ίση με $\varepsilon = 10^{-5}$). Η υλοποίηση της μεθόδου Newton – Raphson έχει πραγματοποιηθεί στην Επίλυση της Άσκησης 1.3 της 1^{ης} Σειράς Ασκήσεων, στο αρχείο τύπου m-function με όνομα “NRMethod.m”, και αξιοποιείται στον κώδικα υλοποίησης της Πεπλεγμένης μεθόδου Euler. Η συνάρτηση NRMethod λαμβάνει ως παραμέτρους το function handle F της συνάρτησης της, εγγενώς μη – γραμμικής, εξίσωσης του (μη – γραμμικού) συστήματος ($\mathbf{F}(\mathbf{x}) = \mathbf{0}$), την αρχική εκτίμηση, x_initial, και την παράμετρο ακρίβειας epsilon, και αξιοποιεί πεπερασμένες διαφορές προς εύρεση του Ιακωβιανού πίνακα της συνάρτησης του συστήματος. Προς αξιοποίηση της μεθόδου, η σχέση (2.5) ανάγεται (για κάθε $i = 1, \dots, N$) σε συνάρτηση μη – γραμμικού μονοδιάστατου συστήματος της μορφής:

$$F(y_{i+1}) \equiv y_{i+1} - y_i - \frac{h}{2} [f(t_i, y_i) + f(t_i, y_{i+1})] = 0 \quad (2.6)$$

όπου οι παράμετροι y_i και t_i είναι δεδομένες «σταθερές» σε κάθε βήμα του κύριου βρόγχου for του αλγορίθμου. Η συνάρτηση (2.6) ορίζεται ως function handle στον υπό επεξήγηση κώδικα του “EulerImplicitMethod.m”, και εισάγεται ως όρισμα στην συνάρτηση “NRMethod.m” που υλοποιεί την μέθοδο Newton – Raphson. Ως αρχική εκτίμηση της λύσης, λαμβάνεται η τιμή του y_{i+1} που προκύπτει με την Ρητή Μέθοδο Euler ($y_{\text{new_initial}} = y_{\text{old}} + h \cdot f(t_{\text{old}}, y_{\text{old}})$), ενώ η ακρίβεια epsilon είναι το αντίστοιχο όρισμα της μεθόδου. Κατά αυτόν τον τρόπο, σε κάθε βήμα $i = 1, \dots, N$ του βρόγχου for του προγράμματος, λαμβάνεται η «επόμενη» τιμή της λύσης y_{i+1} , η οποία είναι λύση της (2.6), και ισοδύναμα της (2.5), εντός της επιλεγόμενης ακρίβειας epsilon. Ο πλήρης πηγαίος κώδικας που υλοποιεί τα παραπάνω παρουσιάζεται παρακάτω, ενώ παρατίθεται και η συνάρτηση “NRMethod.m” υλοποίησης της μεθόδου Newton – Raphson που έχει αναλυθεί και επεξηγηθεί στην Άσκηση 1.3 της 1^{ης} Σειράς Ασκήσεων.

ΣΗΜΕΙΩΣΗ: Τονίζεται ότι για την ομαλή λειτουργία του υπό υλοποίηση αρχείου τύπου m-function με όνομα “EulerImplicitMethod.m”, είναι απαραίτητο το χρησιμοποιούμενο αρχείο m-function με όνομα “NRMethod.m” να βρίσκεται στον ίδιο φάκελο εργασίας (current working directory), ειδάλλως η Matlab δεν μπορεί να αναγνωρίσει, να ανοίξει και να εκτελέσει την αντίστοιχη συνάρτηση, και το όλο πρόγραμμα τερματίζει με σφάλμα (exception error).

MATLAB FUNCTION M-FILE (EulerImplicitMethod.m):

```
function [t,y] = EulerImplicitMethod(f, t_initial, t_final, y0, h, epsilon)
% Function EulerImplicitMethod.m
%-----
% Euler/Heun's Implicit method for I.V.P. with step h:
% { y'(t) = f(t,y), y0 = y(ti)}, t_initial<=t<=t_final }
% Usage of trapezoidal rule for numerical integration
% with tolerance epsilon. The method is iterative and
```

```
% uses Newton-Raphson method (function NRMethod.m).
% -----
% CAUTION : NRMethod.m must be in the same working directory
%           with this m-function file (!)
% -----
% Input : - function handle f for the slope.
%         - initial time parameter t_initial.
%         - final   time parameter t_final.
%         - initial condition y0.
%         - step size h.
%         - termination parameter epsilon.
%           (use of "weighted" infinity norm)
%           (no. of partition intervals: N = (t_final-t_initial)/h)
% Output: - vector y = sol. of I.V.P.      ((N+1) x 1).
%         - vector t = partition node values ((N+1) x 1).
%-----

% --- Input validity ---
% Ensure function handle f is properly defined:
while(true)
    try
        f(rand(1), rand(1));
        break;
    catch
        error('Function handle f must take two scalar inputs (t,y).');
    end
end
% Ensure t_initial, t_final are properly defined:
if(length(t_initial) ~= 1)
    error('t_initial must be a number');
elseif(length(t_final) ~= 1)
    error('t_final must be a number');
end
% Ensure epsilon is positive:
if(epsilon <= 0)
    error('Termination parameter epsilon must be positive.');
```

```
end
% --- Euler's Implicit method implementation ---
% -----
% --- Initialization ---
% Time domain partition nodes:
t = (t_initial : h : t_final)'; % t_intial=t(1), t_final=t(N+1)
N = floor((t_final-t_initial)/h); % No. of partition intervals N.
% Solution vector initialization:
y = zeros(N+1,1);
y(1) = y0; % Initial condition y(1)=y0.
% --- Main loop ---
for i = 1:N
    % Initialization of temporary loop variables:
    t_old = t(i);
    t_new = t(i+1);
    y_old = y(i);
    % --- Usage of Newton-Raphson method ---
    % -----
    % Initial guess using Euler's explicit method:
    y_new_initial = y_old + h * f(t_old, y_old);
    % -----
    % Newton-Raphson method for y(i+1):
```

```

% Definition of 1x1 system F(x) = 0 from the defining
% trapezoidal rule of Implicit Euler's method:
% y_new = y_old + (h/2)*(f(t_old, y_old) + f(t_new,y_new))
% -----
F = @(y_new) y_new - y_old - (h/2)*(f(t_old, y_old) + f(t_new,y_new));
% -----
% Value of y(i+1) (using NRMethod.m, see Ex1.3):
% -----
y(i+1) = NRMethod(F, y_new_initial, epsilon);
end
end

```

MATLAB FUNCTION M-FILE (NRMethod.m):

```

function [x_final, iter] = NRMethod(F, x_initial, epsilon)
% Function NRMethod.m
%-----
% Newton-Raphson's method for non-linear system F(x) = 0.
% (--- The Jacobian matrix is calculated numerically ---)
%
% Input : - function handle F for the system of equations.
%         - initial guess x_initial          (n x 1).
%         - termination parameter epsilon    (1 x 1).
%         (use of "weighted" infinity norm)
% Output: - vector x_final = sol. of (F(x)=0) (n x 1).
%         - integer iter = no. of iterations (1 x 1).
%-----

% --- System dimension definition ---
n = length(x_initial);

% --- Input validity ---
% Ensure epsilon is positive:
if(epsilon <= 0)
    error('Termination parameter epsilon must be positive.');
```

```

end
% Ensure x_initial is in column form :
x_initial = x_initial(:);
% Ensure F is properly sized :
while(true)
    try
        F(rand(n,1));
        break;
    catch
        error('Function handle F and x_initial are incompatible.');
```

```

    end
end

% --- Newton-Raphson method implementation ---
% -----
% --- Temporary and Input variables initialization ---
% Initializations:
J_temp = zeros(n,n);
x_temp = x_initial;
x_final = x_temp;
iter = 0;
% Termination Ratio ("Weighted" infinity norm):
terminationRatio = 1 + epsilon;

```

```
% Perturbation step size for finite differences:
h = 1e-8;
% -----
% --- Main Iterative Loop ---
while(terminationRatio >= epsilon)
    % --- 1. Calculation of F(x(k)) ---
    F_temp = F(x_temp);
    % Ensure column form (F is a function handle):
    F_temp = F_temp(:);
    % --- 2. Calculation of J(x(k)) ---
    for j = 1 : n
        xh_step = x_temp;
        xh_step(j) = x_temp(j) + h; % Perturbation of j-th variable.
        Fh_step = F(xh_step);
        Fh_step = Fh_step(:); % Ensure column form.
        % Finite difference formula [ dFi/dxj ~ (Fi(xj+h)-Fi(xj))/h ]:
        J_temp(:,j) = (Fh_step - F_temp)/h;
    end
    % --- 3. Update solution and temp. parameters ---
    % Check if J_temp is singular:
    if(det(J_temp) == 0)
        disp('Method terminated because J became singular. ');
        break;
    end
    % F_temp is ensured to be in column form, so:
    x_final = x_temp - J_temp\F_temp;
    terminationRatio = norm(x_final-x_temp, "inf") / norm(x_final, "inf");
    iter = iter + 1;
    x_temp = x_final;
end
% -----
end
```

Βάσει της παραπάνω συνάρτησης “EulerImplicitMethod.m”, η επίλυση του Π.Α.Τ. πραγματοποιείται άμεσα, και η διαδικασία είναι όμοια με αυτή του εδαφίου 1 παραπάνω. Στο παρακάτω αρχείο (m-script file), παρουσιάζεται τόσο η επίλυση του δεδομένου Π.Α.Τ. (2.1), όσο και ο τρόπος υπολογισμού των σφαλμάτων της επίλυσής του με την Πεπλεγμένη μέθοδο Euler, σε σχέση με την δεδομένη αναλυτική λύση. Τα αποτελέσματα επίλυσης, καθώς και η αντίστοιχη αναλυτική λύση, παρουσιάζονται και γραφικά. Στο m-script αρχείο, μετά τον ορισμό όλων των απαραίτητων παραμέτρων του προς επίλυση Π.Α.Τ. (2.1), ορίζεται ένα διάνυσμα **error_EI**, διαστάσεων $(N + 1) \times 1$, στο οποίο αποθηκεύονται τα απόλυτα σφάλματα μεταξύ των τιμών της αναλυτικής λύσης (η οποία ορίζεται προηγουμένως στο πρόγραμμα ως κατάλληλο function handle, και οι τιμές της οποίας αποθηκεύονται στο διάνυσμα **y_anal**) στους κόμβους της θεωρούμενης διαμέρισης και των τιμών που προκύπτουν από την Πεπλεγμένη μέθοδο Euler (οι οποίες αποθηκεύονται στο διάνυσμα **y_EI**). Η παρουσίαση των τιμών των διανυσμάτων αυτών στο Command Window γίνεται, για λόγους εύκολης εποπτείας και εύκολης αντιγραφής, μέσω της συνάρτησης **table()** της Matlab. Επιπλέον, οι τιμές των διανυσμάτων στήλης **y_anal** και **y_EI** σχεδιάζονται στο ίδιο διάγραμμα με την εντολή **plot** της Matlab, όπου αξιοποιούνται και διάφοροι «διακόπτες» (switches) για πιο ευπαρουσίαστη γραφική παράσταση (ειδικότερα, η αναλυτική λύση παρουσιάζεται με μία συνεχή γραμμή, χρώματος μπλε,

ενώ η λύση από την Πεπλεγμένη μέθοδο Euler με σημεία, που συμβολίζονται με «×», χρώματος κόκκινο).

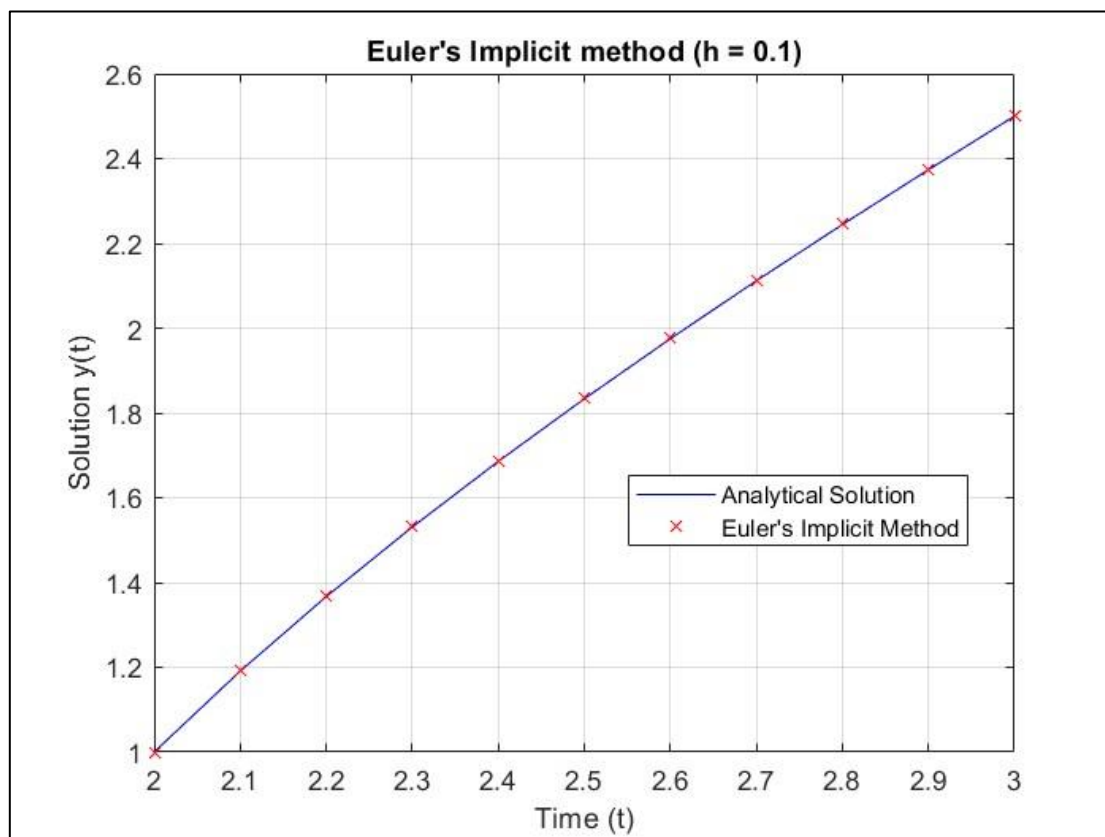
MATLAB SCRIPT M-FILE (EulerImplicitMethodEx2_1_Script.m):

```
clear;
clc;
% --- CFD Exercise 2.1 ---
% --- Part 2 ---
% Solution script (using EulerImplicitMethod.m)
%
% -----
% --- DEFINITION OF PARAMETERS ---
% -----
% --- Definition of function handle ---
f = @(t,y) 1 + (t-y).^2;
% --- Definition of t_initial, t_final ---
t_initial = 2; t_final = 3;
% --- Definition of y0 ---
y0 = 1;
% --- Definition of partition step h ---
h = 0.1;
% --- Definition of accuracy epsilon ---
epsilon = 1e-5;
% --- Definition of number N of intervals ---
N = floor((t_final-t_initial)/h);
% --- Definition of node number vector ---
i = (1:(N+1))';
% --- Definition of time values vector ---
% (this vector will be overwritten below, but it is the same
% for given h, t_initial and t_final)
t = (t_initial : h : t_final)';
% --- Definition of analytical solution function handle ---
y_analytical = @(t) t + (1-t).^-1;
% --- Definition of analytical solution at partition nodes ---
y_anal = y_analytical(t);
%
% -----
% --- SOLUTION OF I.V.P. ---
% -----
% --- Euler's Implicit method solution ---
[t,y_EI] = EulerImplicitMethod(f,t_initial,t_final,y0,h,epsilon);
% --- Error Calculation ---
error_EI = abs(y_EI - y_anal);
% --- Definition of table for data output ---
T = table(i, t, y_anal, y_EI, error_EI);
disp(T);
% --- Plot solution and analytical values vs. t values ---
figure;
plot(t, y_anal, 'b-', 'LineWidth', 0.5); hold on;
plot(t, y_EI, 'rx', 'LineWidth', 0.5);
grid on;
xlabel('Time (t)');
ylabel('Solution y(t)');
title('Euler''s Implicit method (h = 0.1)');
legend('Analytical Solution', 'Euler''s Implicit Method', 'Location',
'Best');
% End of script.
```

Τρέχοντας το παραπάνω m-script αρχείο, λαμβάνουμε στο τερματικό του Matlab την λύση του δεδομένου Π.Α.Τ. με την Πεπλεγμένη μέθοδο Euler (σε μορφή εκτύπωσης που καθορίζεται από την συνάρτηση `table`), τα αποτελέσματα της οποίας αναγράφονται στον Πίνακα II. Λαμβάνουμε επίσης την γραφική παράσταση της αναλυτικής λύσης, καθώς και των αριθμητικά υπολογιζόμενων με την Πεπλεγμένη μέθοδο Euler σημείων, στους κόμβους της διαμέρισης (Βλ. Διάγραμμα II παρακάτω).

Επίλυση Π.Α.Τ. – Πεπλεγμένη Μέθοδος Euler				
i	Τιμές t	Αναλυτική Λύση $y_{\text{analytical}}$	Πεπλ. M. Euler y_{EI}	Σφάλμα Πεπλ. M. Euler error_{EI}
1	2.0	1.0000	1.0000	0.0000
2	2.1	1.1909	1.1913	0.00037879
3	2.2	1.3667	1.3672	0.00058278
4	2.3	1.5308	1.5315	0.00068687
5	2.4	1.6857	1.6864	0.0007327
6	2.5	1.8333	1.8341	0.00074419
7	2.6	1.9750	1.9757	0.00073547
8	2.7	2.1118	2.1125	0.00071507
9	2.8	2.2444	2.2451	0.00068822
10	2.9	2.3738	2.3743	0.00065814
11	3.0	2.5000	2.5006	0.00062683

Πίνακας II.



Διάγραμμα II.

Σημειώνεται ότι τα δεδομένα του Πίνακα II αναγράφονται με 4 δεκαδικά ψηφία, όπως εμφανίζονται και στο Command Window, υπό την εντολή `format default`, παρόλο που η Matlab αποθηκεύει τις τιμές των αποτελεσμάτων σε περισσότερα δεκαδικά ψηφία (16 σημαντικά ψηφία). Ωστόσο, η παρεχόμενη ακρίβεια της Πεπλεγμένης μεθόδου Euler (καθολικό σφάλμα $O(h^2)$ – μέθοδος δεύτερης τάξης), για την επιλεγόμενη τιμή βήματος $h = 0.1$ αποδίδει *προσεγγιστικά* ακρίβεια στο δεύτερο δεκαδικό ψηφίο, γεγονός που φαίνεται άμεσα και από τις τιμές του Πίνακα II, καθώς η επίλυση μέσω της μεθόδου ταυτίζεται με την αναλυτική λύση *το πολύ* στο δεύτερο δεκαδικό ψηφίο. Επομένως, η «αποκοπή» των δεκαδικών ψηφίων (η οποία *δεν* έχει εφαρμοστεί στα σημεία του Διαγράμματος II), εκτός του ότι πραγματοποιείται για λόγους ευκρίνειας των αποτελεσμάτων, είναι και δικαιολογημένη λόγω της παρεχόμενης ακρίβειας της λύσης από τον αλγόριθμο.

3. Συμβαδίζοντας με την γενική λογική των προηγούμενων εδαφίων, υλοποιείται σε γλώσσα Matlab αλγόριθμος Runge – Kutta 2^{ης} τάξης, ακολουθώντας την αλγοριθμική διαδικασία των Σημειώσεων του μαθήματος. Ο πηγαίος κώδικας γράφεται, και αποθηκεύεται, σε ένα αρχείο Matlab τύπου συνάρτησης (m-function file) με όνομα “RK2Method.m”, και παρατίθεται παρακάτω. Η συνάρτηση δέχεται ως παραμέτρους την συνάρτηση «κλίσης», f , του δεξιού μέλους της εξίσωσης του Π.Α.Τ. (σε τύπο function handle), το χρονικό εύρος, μέσω των $t_{initial}$ και t_{final} , την αρχική συνθήκη y_0 , στο σημείο $t_{initial}$, καθώς και το μήκος, h , των διαστημάτων της ομοιόμορφης διαμέρισης που πραγματοποιείται στο πεδίο ορισμού του Π.Α.Τ.

Αρχικά, γίνεται έλεγχος της ορθότητας μορφής των δεδομένων εισόδου, ο οποίος είναι πανομοιότυπος με τον έλεγχο στην υλοποίηση της Ρητής μεθόδου Euler. Μετά τον έλεγχο ορθότητας των δεδομένων εισόδου, η συνάρτηση υλοποιεί τον αλγόριθμο της Runge – Kutta 2^{ης} τάξης. Προς αυτό τον σκοπό, αρχικοποιείται το διάνυσμα των χρονικών τιμών που αντιστοιχούν στην ομοιόμορφη διαμέριση του διαστήματος ορισμού του (2.1), και εξασφαλίζεται ότι το προκύπτον διάνυσμα είναι μία στήλη (με τον τόνο “’” στο τέλος της αντίστοιχης εντολής), ενώ ορίζεται και ο αριθμός των διαστημάτων της διαμέρισης, N , βάσει της (2.2). Το διάνυσμα με τις τιμές της επίλυσης του Π.Α.Τ., αρχικοποιείται σε μηδενικές τιμές, και ορίζεται να έχει διάσταση $(N + 1) \times 1$, όπου N ο αριθμός των διαστημάτων διαμέρισης, ενώ επιβάλλεται και η αρχική συνθήκη του Π.Α.Τ. στην τιμή του $y(1)$. Στην συνέχεια, υλοποιείται ο επαναληπτικός βρόγχος της μεθόδου, όπου για κάθε τιμή της μεταβλητής i από 1 έως και N , οι τιμές της λύσης του Π.Α.Τ. υπολογίζονται βάσει των σχέσεων:

$$\left\{ \begin{array}{l} k_1 = f(t_i, y_i) \\ k_2 = f(x_{i+1}, y_i + hk_1) \\ y_{i+1} = y_i + \frac{h}{2} \cdot (k_1 + k_2) \end{array} \right\} \quad i = 1, 2, \dots, N, \quad y(1) \equiv y_0 \quad (2.7)$$

η οποία ορίζει και την μέθοδο Runge – Kutta 2^{ης} τάξης. Τονίζεται ότι, σε αντίθεση με τον αλγόριθμο που παρουσιάζεται στις Σημειώσεις του μαθήματος, όπου οι δείκτες της διαμέρισης λαμβάνουν τιμές στο εύρος ακεραίων $\{0, 1, \dots, N\}$, στην υλοποίηση της μεθόδου, οι δείκτες κυμαίνονται στο διάστημα ακεραίων $\{1, 2, \dots, N + 1\}$. Ο πλήρης πηγαίος κώδικας που υλοποιεί τα παραπάνω παρουσιάζεται παρακάτω.

MATLAB FUNCTION M-FILE (RK2Method.m):

```
function [t,y] = RK2Method(f, t_initial, t_final, y0, h)
% Function RK2Method.m
%-----
% Runge-Kutta of 2nd order method for I.V.P. with step h:
% { y'(t) = f(t,y), y0 = y(ti), t_initial<=t<=t_final }
%
% Input : - function handle f for the slope.
%         - initial time parameter t_initial.
%         - final   time parameter t_final.
%         - initial condition y0.
%         - step size h
%         (no. of partition intervals: N = (t_final-t_initial)/h)
% Output: - vector y = sol. of I.V.P.      ((N+1) x 1).
%         - vector t = partition node values ((N+1) x 1).
%-----

% --- Input validity ---
% Ensure function handle f is properly defined:
while(true)
    try
        f(rand(1), rand(1));
        break;
    catch
        error('Function handle f must take two scalar inputs (t,y).');
    end
end

% Ensure t_initial, t_final are properly defined:
if(length(t_initial) ~= 1)
    error('t_initial must be a number');
elseif(length(t_final) ~= 1)
    error('t_final must be a number');
end

% --- Runge-Kutta of 2nd order method implementation ---
% -----
% --- Initialization ---
% Time domain partition nodes:
t = (t_initial : h : t_final)'; % t_intial=t(1), t_final=t(N+1)
N = floor((t_final-t_initial)/h); % No. of partition intervals N.
% Solution vector initialization:
y = zeros(N+1,1);
y(1) = y0; % Initial condition y(1)=y0.
% --- Main loop ---
for i = 1:N
    k1 = f(t(i),y(i));
    k2 = f(t(i+1),y(i)+h*k1);
    y(i+1) = y(i) + (h/2)*(k1+k2);
end
end
```

Βάσει της παραπάνω συνάρτησης, η επίλυση του Π.Α.Τ. πραγματοποιείται άμεσα. Στο παρακάτω αρχείο (m-script file), παρουσιάζεται τόσο η επίλυση του δεδομένου Π.Α.Τ. (2.1), όσο και ο τρόπος υπολογισμού των σφαλμάτων της επίλυσής του με την μέθοδο Runge – Kutta 2^{ης} τάξης, σε σχέση με την δεδομένη αναλυτική λύση. Τα αποτελέσματα

επίλυσης, καθώς και η αντίστοιχη αναλυτική λύση, παρουσιάζονται και γραφικά. Το παρακάτω m-script αρχείο είναι πανομοιότυπο με αυτά των προηγούμενων εδαφίων, και η επεξήγησή παραλείπεται.

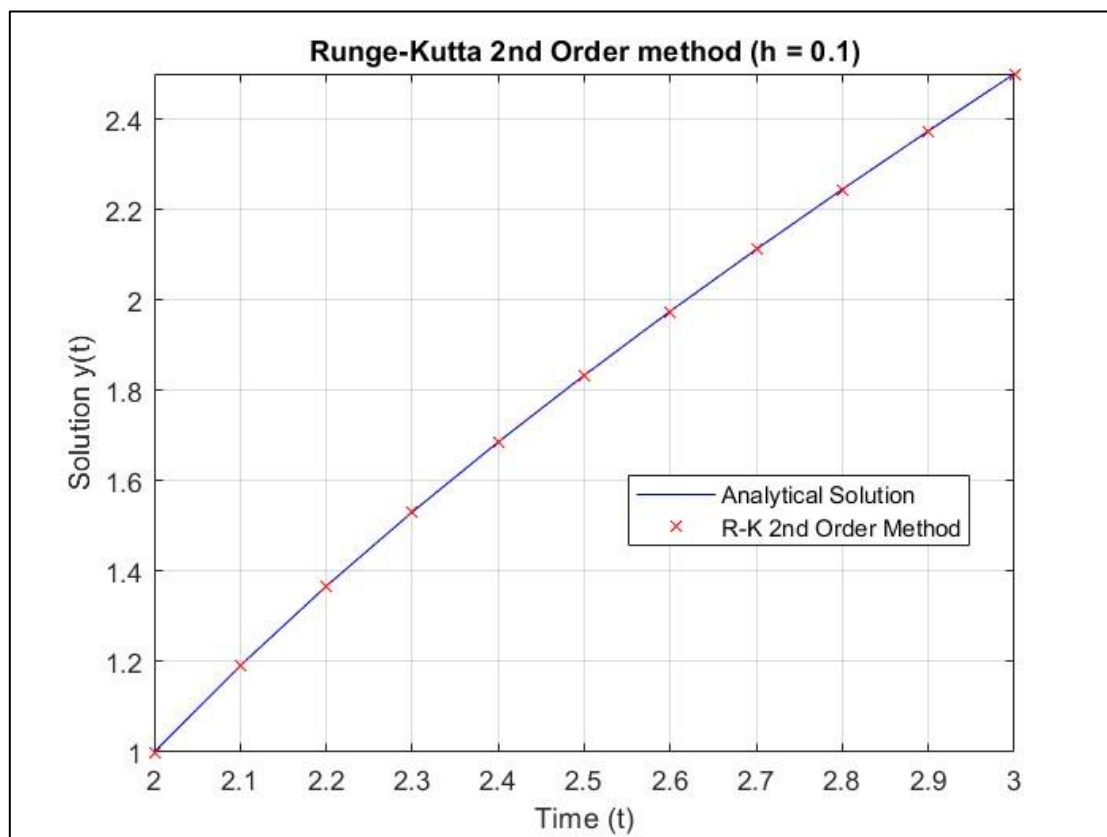
MATLAB SCRIPT M-FILE (RK2MethodEx2_1_Script.m):

```
clear; clc;
% --- CFD Exercise 2.1 ---
%      --- Part 3 ---
% Solution script (using RK2Method.m)
%
% -----
% --- DEFINITION OF PARAMETERS ---
% -----
% --- Definition of function handle ---
f = @(t,y) 1 + (t-y).^2;
% --- Definition of t_initial, t_final ---
t_initial = 2; t_final = 3;
% --- Definition of y0 ---
y0 = 1;
% --- Definition of partition step h ---
h = 0.1;
% --- Definition of accuracy epsilon ---
epsilon = 1e-5;
% --- Definition of number N of intervals ---
N = floor((t_final-t_initial)/h);
% --- Definition of node number vector ---
i = (1:(N+1))';
% --- Definition of time values vector ---
% (this vector will be overwritten below, but it is the same
%      for given h, t_initial and t_final)
t = (t_initial : h : t_final)';
% --- Definition of analytical solution function handle ---
y_analytical = @(t) t + (1-t).^-1;
% --- Definition of analytical solution at partition nodes ---
y_anal = y_analytical(t);
%
% -----
% --- SOLUTION OF I.V.P. ---
% -----
% --- Runge-Kutta 2nd order method solution ---
[t,y_RK2] = RK2Method(f,t_initial,t_final,y0,h);
% --- Error Calculation ---
error_RK2 = abs(y_RK2 - y_anal);
% --- Definition of table for data output ---
T = table(i, t, y_anal, y_RK2, error_RK2);
disp(T);
% --- Plot solution and analytical values vs. t values ---
figure;
plot(t, y_anal, 'b-', 'LineWidth', 0.5); hold on;
plot(t, y_RK2, 'rx', 'LineWidth', 0.5);
grid on;
xlabel('Time (t)');
ylabel('Solution y(t)');
yticks(1:0.2:2.6);
title('Runge-Kutta 2nd Order method (h = 0.1)');
legend('Analytical Solution', 'R-K 2nd Order Method', 'Location', 'Best');
% End of script.
```

Τρέχοντας το παραπάνω m-script αρχείο, λαμβάνουμε στο τερματικό του Matlab την λύση του δεδομένου Π.Α.Τ. με την μέθοδο Runge – Kutta 2^{ης} τάξης (σε μορφή εκτύπωσης που καθορίζεται από την συνάρτηση `table`), τα αποτελέσματα της οποίας αναγράφονται στον Πίνακα III. Λαμβάνουμε επίσης την γραφική παράσταση της αναλυτικής λύσης, καθώς και της αριθμητικά υπολογιζόμενων με την μέθοδο σημείων, στους κόμβους της διαμέρισης (Βλ. Διάγραμμα III παρακάτω).

Επίλυση Π.Α.Τ. – Μέθοδος Runge-Kutta 2 ^{ης} Τάξης				
i	Τιμές t	Αναλυτική Λύση $y_{\text{analytical}}$	Μέθοδος RK2 y_{RK2}	Σφάλμα M. RK2 $\text{error}_{\text{RK2}}$
1	2.0	1.0000	1.0000	0.00000
2	2.1	1.1909	1.1905	0.00040909
3	2.2	1.3667	1.3660	0.00062882
4	2.3	1.5308	1.5300	0.00074038
5	2.4	1.6857	1.6849	0.00078896
6	2.5	1.8333	1.8325	0.0008005
7	2.6	1.9750	1.9742	0.00079034
8	2.7	2.1118	2.1110	0.00076769
9	2.8	2.2444	2.2437	0.00073819
10	2.9	2.3737	2.3730	0.00070532
11	3.0	2.5000	2.4993	0.00067122

Πίνακας III.



Διάγραμμα III.

Σημειώνεται ότι τα δεδομένα του Πίνακα III αναγράφονται με 4 δεκαδικά ψηφία, όπως εμφανίζονται και στο Command Window, υπό την εντολή `format default`, παρόλο που η Matlab αποθηκεύει τις τιμές των αποτελεσμάτων σε περισσότερα δεκαδικά ψηφία (16 σημαντικά ψηφία). Ωστόσο, η παρεχόμενη ακρίβεια της μεθόδου Runge – Kutta 2^{ης} τάξης (καθολικό σφάλμα $O(h^2)$), για την επιλεγόμενη τιμή βήματος $h = 0.1$ αποδίδει *προσεγγιστικά* ακρίβεια στο δεύτερο δεκαδικό ψηφίο, γεγονός που φαίνεται άμεσα και από τις τιμές του Πίνακα III, καθώς η επίλυση μέσω της μεθόδου ταυτίζεται με την αναλυτική λύση *το πολύ* στο δεύτερο δεκαδικό ψηφίο. Επομένως, η «αποκοπή» των δεκαδικών ψηφίων (η οποία *δεν έχει* εφαρμοστεί στα σημεία του Διαγράμματος I), εκτός του ότι πραγματοποιείται για λόγους ευκρίνειας των αποτελεσμάτων, είναι και δικαιολογημένη λόγω της παρεχόμενης ακρίβειας της λύσης από τον αλγόριθμο.

4. Επαναλαμβάνοντας τα παραπάνω, υλοποιείται σε γλώσσα Matlab αλγόριθμος Runge – Kutta 3^{ης} τάξης, ακολουθώντας την αλγοριθμική διαδικασία των Σημειώσεων του μαθήματος. Ο πηγαίος κώδικας γράφεται, και αποθηκεύεται, σε ένα αρχείο Matlab τύπου συνάρτησης (m-function file) με όνομα “RK3Method.m”, και παρατίθεται παρακάτω. Η συνάρτηση δέχεται ως παραμέτρους την συνάρτηση «κλίσης», `f`, του δεξιού μέλους της εξίσωσης του Π.Α.Τ. (σε τύπο function handle), το χρονικό εύρος, μέσω των `t_initial` και `t_final`, την αρχική συνθήκη y_0 , στο σημείο `t_initial`, καθώς και το μήκος, h , των διαστημάτων της ομοιόμορφης διαμέρισης που πραγματοποιείται στο πεδίο ορισμού του Π.Α.Τ.

Αρχικά, γίνεται έλεγχος της ορθότητας μορφής των δεδομένων εισόδου, ο οποίος είναι πανομοιότυπος με τον έλεγχο στην υλοποίηση της Ρητής μεθόδου Euler. Μετά τον έλεγχο ορθότητας των δεδομένων εισόδου, η συνάρτηση υλοποιεί τον αλγόριθμο της Runge – Kutta 3^{ης} τάξης. Προς αυτό τον σκοπό, αρχικοποιείται το διάνυσμα των χρονικών τιμών που αντιστοιχούν στην ομοιόμορφη διαμέριση του διαστήματος ορισμού του (2.1), και εξασφαλίζεται ότι το προκύπτον διάνυσμα είναι μία στήλη (με τον τόνο “’” στο τέλος της αντίστοιχης εντολής), ενώ ορίζεται και ο αριθμός των διαστημάτων της διαμέρισης, N , βάσει της (2.2). Το διάνυσμα με τις τιμές της επίλυσης του Π.Α.Τ., αρχικοποιείται σε μηδενικές τιμές, και ορίζεται να έχει διάσταση $(N + 1) \times 1$, όπου N ο αριθμός των διαστημάτων διαμέρισης, ενώ επιβάλλεται και η αρχική συνθήκη του Π.Α.Τ. στην τιμή του $y(1)$. Στην συνέχεια, υλοποιείται ο επαναληπτικός βρόγχος της μεθόδου, όπου για κάθε τιμή της μεταβλητής i από 1 έως και N , οι τιμές της λύσης του Π.Α.Τ. υπολογίζονται βάσει των σχέσεων:

$$\left\{ \begin{array}{l} k_1 = f(t_i, y_i) \\ k_2 = f\left(x_i + \frac{h}{2}, y_i + \frac{h}{2}k_1\right) \\ k_3 = f(x_{i+1}, y_i + hk_2) \\ y_{i+1} = y_i + \frac{h}{6} \cdot (k_1 + 4k_2 + k_3) \end{array} \right\} \quad i = 1, 2, \dots, N, \quad y(1) \equiv y_0 \quad (2.8)$$

η οποία ορίζει και την μέθοδο Runge – Kutta 3^{ης} τάξης (η οποία πηγάζει από τον 1ο κανόνα του Simpon). Τονίζεται ότι, σε αντίθεση με τον αλγόριθμο που παρουσιάζεται στις Σημειώσεις του μαθήματος, όπου οι δείκτες της διαμέρισης λαμβάνουν τιμές στο εύρος ακεραίων $\{0, 1, \dots, N\}$, στην υλοποίηση της μεθόδου, οι δείκτες κυμαίνονται στο

διάστημα ακεραίων $\{1, 2, \dots, N + 1\}$. Ο πλήρης πηγαίος κώδικας που υλοποιεί τα παραπάνω παρουσιάζεται παρακάτω.

MATLAB FUNCTION M-FILE (RK3Method.m):

```
function [t,y] = RK3Method(f, t_initial, t_final, y0, h)
% Function RK3Method.m
%-----
% Runge-Kutta of 3rd order method for I.V.P. with step h:
% { y'(t) = f(t,y), y0 = y(ti), t_initial<=t<=t_final }
%
% Input : - function handle f for the slope.
%         - initial time parameter t_initial.
%         - final   time parameter t_final.
%         - initial condition y0.
%         - step size h
%         (no. of partition intervals: N = (t_final-t_initial)/h)
% Output: - vector y = sol. of I.V.P.      ((N+1) x 1).
%         - vector t = partition node values ((N+1) x 1).
%-----

% --- Input validity ---
% Ensure function handle f is properly defined:
while(true)
    try
        f(rand(1), rand(1));
        break;
    catch
        error('Function handle f must take two scalar inputs (t,y).');
    end
end
% Ensure t_initial, t_final are properly defined:
if(length(t_initial) ~= 1)
    error('t_initial must be a number');
elseif(length(t_final) ~= 1)
    error('t_final must be a number');
end

% --- Runge-Kutta of 3rd order method implementation ---
% -----
% --- Initialization ---
% Time domain partition nodes:
t = (t_initial : h : t_final)'; % t_intial=t(1), t_final=t(N+1)
N = floor((t_final-t_initial)/h); % No. of partition intervals N.
% Solution vector initialization:
y = zeros(N+1,1);
y(1) = y0; % Initial condition y(1)=y0.
% --- Main loop ---
for i = 1:N
    k1 = f(t(i),y(i));
    k2 = f(t(i)+h/2,y(i)+(h/2)*k1);
    k3 = f(t(i+1),y(i)+h*k2);
    y(i+1) = y(i) + (h/6)*(k1+4*k2+k3);
end
end
```

Βάσει της παραπάνω συνάρτησης, η επίλυση του Π.Α.Τ. πραγματοποιείται άμεσα μέσω του παρακάτω m-script αρχείου, το οποίο είναι πανομοιότυπο με αυτά των προηγούμενων εδαφίων, και η επεξήγησή του παραλείπεται.

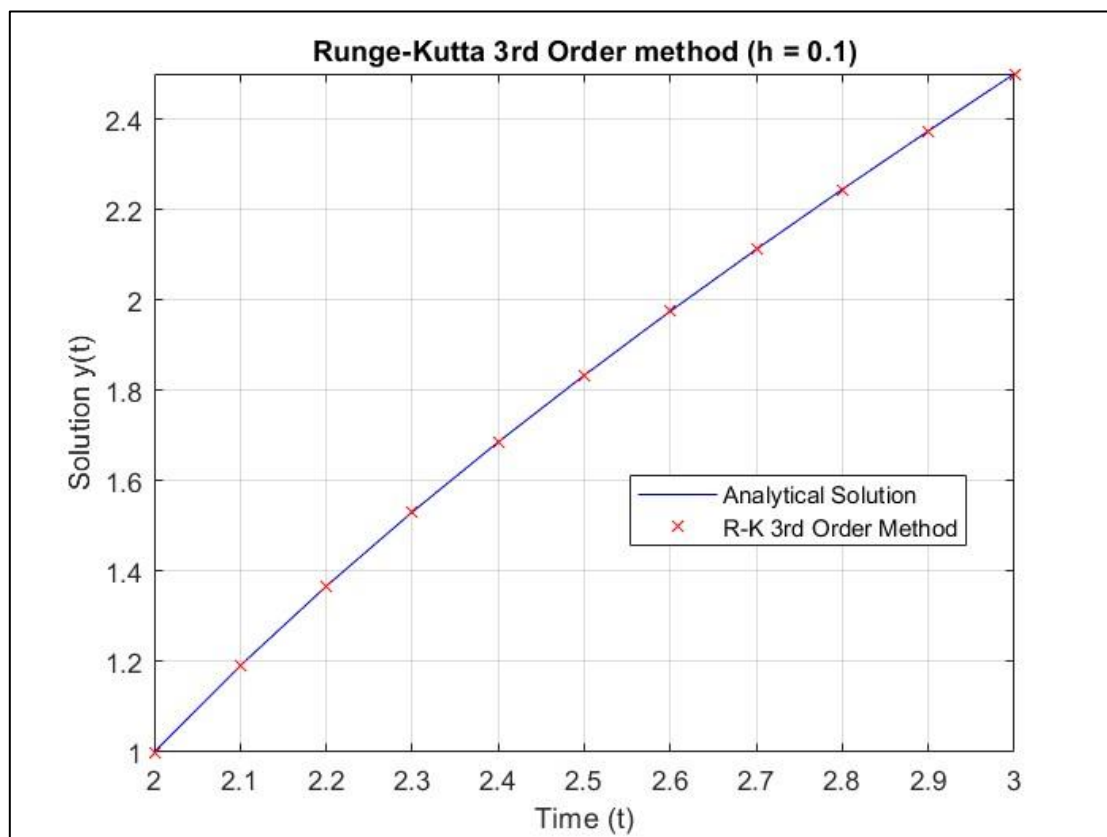
MATLAB SCRIPT M-FILE (RK3MethodEx2_1_Script.m):

```
clear; clc;
% --- CFD Exercise 2.1 ---
%      --- Part 4 ---
% Solution script (using RK3Method.m)
%
% -----
% --- DEFINITION OF PARAMETERS ---
% -----
% --- Definition of function handle ---
f = @(t,y) 1 + (t-y).^2;
% --- Definition of t_initial, t_final ---
t_initial = 2; t_final = 3;
% --- Definition of y0 ---
y0 = 1;
% --- Definition of partition step h ---
h = 0.1;
% --- Definition of accuracy epsilon ---
epsilon = 1e-5;
% --- Definition of number N of intervals ---
N = floor((t_final-t_initial)/h);
% --- Definition of node number vector ---
i = (1:(N+1))';
% --- Definition of time values vector ---
% (this vector will be overwritten below, but it is the same
%      for given h, t_initial and t_final)
t = (t_initial : h : t_final)';
% --- Definition of analytical solution function handle ---
y_analytical = @(t) t + (1-t).^-1;
% --- Definition of analytical solution at partition nodes ---
y_anal = y_analytical(t);
%
% -----
% --- SOLUTION OF I.V.P. ---
% -----
% --- Runge-Kutta 3rd order method solution ---
[t,y_RK3] = RK3Method(f,t_initial,t_final,y0,h);
% --- Error Calculation ---
error_RK3 = abs(y_RK3 - y_anal);
% --- Definition of table for data output ---
T = table(i, t, y_anal, y_RK3, error_RK3);
disp(T);
% --- Plot solution and analytical values vs. t values ---
figure;
plot(t, y_anal, 'b-', 'LineWidth', 0.5); hold on;
plot(t, y_RK3, 'rx', 'LineWidth', 0.5);
grid on;
xlabel('Time (t)');
ylabel('Solution y(t)');
yticks(1: 0.2 : 2.6);
title('Runge-Kutta 3rd Order method (h = 0.1)');
legend('Analytical Solution', 'R-K 3rd Order Method', 'Location', 'Best');
% End of script.
```

Τρέχοντας το παραπάνω m-script αρχείο, λαμβάνουμε στο τερματικό του Matlab την λύση του δεδομένου Π.Α.Τ. με την μέθοδο Runge – Kutta 3^{ης} τάξης (σε μορφή εκτύπωσης που καθορίζεται από την συνάρτηση `table`), τα αποτελέσματα της οποίας αναγράφονται στον Πίνακα IV. Λαμβάνουμε επίσης την γραφική παράσταση της αναλυτικής λύσης, καθώς και της αριθμητικά υπολογιζόμενων με την μέθοδο σημείων, στους κόμβους της διαμέρισης (Βλ. Διάγραμμα IV παρακάτω).

Επίλυση Π.Α.Τ. – Μέθοδος Runge-Kutta 3 ^{ης} Τάξης				
i	Τιμές t	Αναλυτική Λύση $y_{\text{analytical}}$	Μέθοδος RK3 y_{RK3}	Σφάλμα M. RK3 $\text{error}_{\text{RK3}}$
1	2.0	1.0000	1.0000	0.00000
2	2.1	1.1909	1.1906	0.00028167
3	2.2	1.3667	1.3662	0.00043217
4	2.3	1.5308	1.5303	0.00050809
5	2.4	1.6857	1.6852	0.00054075
6	2.5	1.8333	1.8328	0.0005481
7	2.6	1.9750	1.9745	0.00054066
8	2.7	2.1118	2.1112	0.00052477
9	2.8	2.2444	2.2439	0.00050427
10	2.9	2.3737	2.3732	0.00048154
11	3.0	2.5000	2.4995	0.00045803

Πίνακας IV.



Διάγραμμα IV.

Σημειώνεται ότι τα δεδομένα του Πίνακα IV αναγράφονται με 4 δεκαδικά ψηφία, όπως εμφανίζονται και στο Command Window, υπό την εντολή `format default`, παρόλο που η Matlab αποθηκεύει τις τιμές των αποτελεσμάτων σε περισσότερα δεκαδικά ψηφία (16 σημαντικά ψηφία). Ωστόσο, η παρεχόμενη ακρίβεια της μεθόδου Runge – Kutta 3^{ης} τάξης (καθολικό σφάλμα $O(h^3)$), για την επιλεγόμενη τιμή βήματος $h = 0.1$ αποδίδει *προσεγγιστικά* ακρίβεια στο τρίτο δεκαδικό ψηφίο, γεγονός που φαίνεται άμεσα και από τις τιμές του Πίνακα IV, καθώς η επίλυση μέσω της μεθόδου ταυτίζεται με την αναλυτική λύση *το πολύ* στο τρίτο δεκαδικό ψηφίο. Επομένως, η «αποκοπή» των δεκαδικών ψηφίων (η οποία δεν έχει εφαρμοστεί στα σημεία του Διαγράμματος IV), εκτός του ότι πραγματοποιείται για λόγους ευκρίνειας των αποτελεσμάτων, είναι και δικαιολογημένη λόγω της παρεχόμενης ακρίβειας της λύσης από τον αλγόριθμο.

5. Επαναλαμβάνοντας για μία ακόμα φορά τα παραπάνω, υλοποιείται σε γλώσσα Matlab αλγόριθμος Runge – Kutta 4^{ης} τάξης, ακολουθώντας την αλγοριθμική διαδικασία των Σημειώσεων του μαθήματος. Ο πηγαίος κώδικας γράφεται, και αποθηκεύεται, σε ένα αρχείο Matlab τύπου συνάρτησης (m-function file) με όνομα “RK4Method.m”, και παρατίθεται παρακάτω. Η συνάρτηση δέχεται ως παραμέτρους την συνάρτηση «κλίσης», f , του δεξιού μέλους της εξίσωσης του Π.Α.Τ. (σε τύπο function handle), το χρονικό εύρος, μέσω των `t_initial` και `t_final`, την αρχική συνθήκη y_0 , στο σημείο `t_initial`, καθώς και το μήκος, h , των διαστημάτων της ομοιόμορφης διαμέρισης που πραγματοποιείται στο πεδίο ορισμού του Π.Α.Τ.

Αρχικά, γίνεται έλεγχος της ορθότητας μορφής των δεδομένων εισόδου, ο οποίος είναι πανομοιότυπος με τον έλεγχο στην υλοποίηση της Ρητής μεθόδου Euler. Μετά τον έλεγχο ορθότητας των δεδομένων εισόδου, η συνάρτηση υλοποιεί τον αλγόριθμο της Runge – Kutta 4^{ης} τάξης. Προς αυτό τον σκοπό, αρχικοποιείται το διάνυσμα των χρονικών τιμών που αντιστοιχούν στην ομοιόμορφη διαμέριση του διαστήματος ορισμού του (2.1), και εξασφαλίζεται ότι το προκύπτον διάνυσμα είναι μία στήλη (με τον τόνο “’” στο τέλος της αντίστοιχης εντολής), ενώ ορίζεται και ο αριθμός των διαστημάτων της διαμέρισης, N , βάσει της (2.2). Το διάνυσμα με τις τιμές της επίλυσης του Π.Α.Τ., αρχικοποιείται σε μηδενικές τιμές, και ορίζεται να έχει διάσταση $(N + 1) \times 1$, όπου N ο αριθμός των διαστημάτων διαμέρισης, ενώ επιβάλλεται και η αρχική συνθήκη του Π.Α.Τ. στην τιμή του $y(1)$. Στην συνέχεια, υλοποιείται ο επαναληπτικός βρόγχος της μεθόδου, όπου για κάθε τιμή της μεταβλητής i από 1 έως και N , οι τιμές της λύσης του Π.Α.Τ. υπολογίζονται βάσει των σχέσεων:

$$\left\{ \begin{array}{l} k_1 = f(t_i, y_i) \\ k_2 = f\left(x_i + \frac{h}{2}, y_i + \frac{h}{2}k_1\right) \\ k_3 = f\left(x_i + \frac{h}{2}, y_i + \frac{h}{2}k_2\right) \\ k_4 = f(x_{i+1}, y_i + hk_3) \\ y_{i+1} = y_i + \frac{h}{6} \cdot (k_1 + 2k_2 + 2k_3 + k_4) \end{array} \right\} \quad i = 1, 2, \dots, N, \quad y(1) \equiv y_0 \quad (2.8)$$

η οποία ορίζει και την υπό χρήση μέθοδο Runge – Kutta 4^{ης} τάξης. Τονίζεται ότι, σε αντίθεση με τον αλγόριθμο που παρουσιάζεται στις Σημειώσεις του μαθήματος, όπου οι δείκτες της διαμέρισης λαμβάνουν τιμές στο εύρος ακεραίων $\{0, 1, \dots, N\}$, στην

υλοποίηση της μεθόδου, οι δείκτες κυμαίνονται, όπως έχει ξαναειπωθεί, στο διάστημα ακεραίων $\{1, 2, \dots, N + 1\}$. Ο πλήρης πηγαίος κώδικας που υλοποιεί τα παραπάνω παρουσιάζεται παρακάτω.

MATLAB FUNCTION M-FILE (RK4Method.m):

```
function [t,y] = RK4Method(f, t_initial, t_final, y0, h)
% Function RK4Method.m
%-----
% Runge-Kutta of 4th order method for I.V.P. with step h:
% { y'(t) = f(t,y), y0 = y(ti), t_initial<=t<=t_final }
%
% Input : - function handle f for the slope.
%         - initial time parameter t_initial.
%         - final   time parameter t_final.
%         - initial condition y0.
%         - step size h
%         (no. of partition intervals: N = (t_final-t_initial)/h)
% Output: - vector y = sol. of I.V.P.      ((N+1) x 1).
%         - vector t = partition node values ((N+1) x 1).
%-----

% --- Input validity ---
% Ensure function handle f is properly defined:
while(true)
    try
        f(rand(1), rand(1));
        break;
    catch
        error('Function handle f must take two scalar inputs (t,y).');
    end
end

% Ensure t_initial, t_final are properly defined:
if(length(t_initial) ~= 1)
    error('t_initial must be a number');
elseif(length(t_final) ~= 1)
    error('t_final must be a number');
end

% --- Runge-Kutta of 4th order method implementation ---
% -----
% --- Initialization ---
% Time domain partition nodes:
t = (t_initial : h : t_final)'; % t_intial=t(1), t_final=t(N+1)
N = floor((t_final-t_initial)/h); % No. of partition intervals N.
% Solution vector initialization:
y = zeros(N+1,1);
y(1) = y0; % Initial condition y(1)=y0.
% --- Main loop ---
for i = 1:N
    k1 = f(t(i),y(i));
    k2 = f(t(i)+h/2,y(i)+(h/2)*k1);
    k3 = f(t(i)+h/2,y(i)+(h/2)*k2);
    k4 = f(t(i+1),y(i)+h*k3);
    y(i+1) = y(i) + (h/6)*(k1+2*k2+2*k3+k4);
end
end
```

Βάσει της παραπάνω συνάρτησης, η επίλυση του Π.Α.Τ. πραγματοποιείται άμεσα μέσω του παρακάτω m-script αρχείου, το οποίο είναι πανομοιότυπο με αυτά των προηγούμενων εδαφίων, και η επεξήγησή του παραλείπεται.

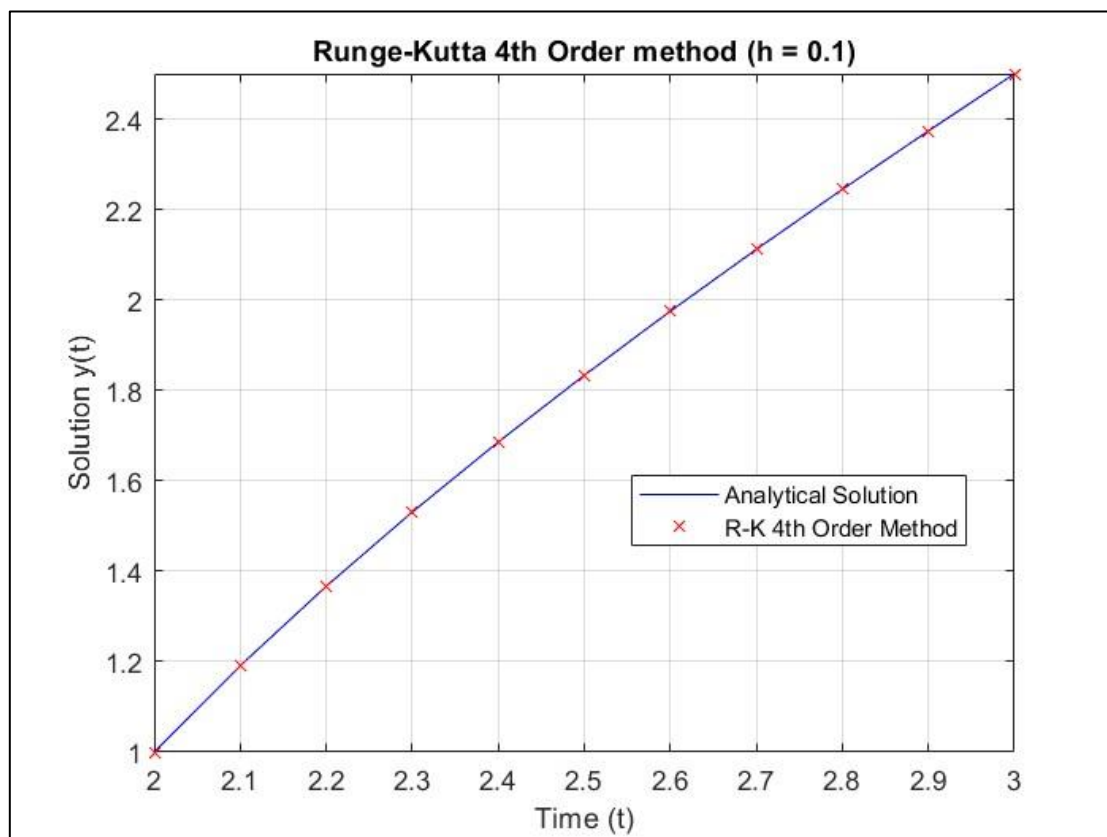
MATLAB SCRIPT M-FILE (RK4MethodEx2_1_Script.m):

```
clear; clc;
% --- CFD Exercise 2.1 ---
%      --- Part 5 ---
% Solution script (using RK4Method.m)
%
% -----
% --- DEFINITION OF PARAMETERS ---
% -----
% --- Definition of function handle ---
f = @(t,y) 1 + (t-y).^2;
% --- Definition of t_initial, t_final ---
t_initial = 2; t_final = 3;
% --- Definition of y0 ---
y0 = 1;
% --- Definition of partition step h ---
h = 0.1;
% --- Definition of accuracy epsilon ---
epsilon = 1e-5;
% --- Definition of number N of intervals ---
N = floor((t_final-t_initial)/h);
% --- Definition of node number vector ---
i = (1:(N+1))';
% --- Definition of time values vector ---
% (this vector will be overwritten below, but it is the same
%      for given h, t_initial and t_final)
t = (t_initial : h : t_final)';
% --- Definition of analytical solution function handle ---
y_analytical = @(t) t + (1-t).^-1;
% --- Definition of analytical solution at partition nodes ---
y_anal = y_analytical(t);
%
% -----
% --- SOLUTION OF I.V.P. ---
% -----
% --- Runge-Kutta 4th order method solution ---
[t,y_RK4] = RK4Method(f,t_initial,t_final,y0,h);
% --- Error Calculation ---
error_RK4 = abs(y_RK4 - y_anal);
% --- Definition of table for data output ---
T = table(i, t, y_anal, y_RK4, error_RK4);
disp(T);
% --- Plot solution and analytical values vs. t values ---
figure;
plot(t, y_anal, 'b-', 'LineWidth', 0.5); hold on;
plot(t, y_RK4, 'rx', 'LineWidth', 0.5);
grid on;
xlabel('Time (t)');
ylabel('Solution y(t)');
yticks(1: 0.2 : 2.6);
title('Runge-Kutta 4th Order method (h = 0.1)');
legend('Analytical Solution', 'R-K 4th Order Method', 'Location', 'Best');
% End of script.
```

Τρέχοντας το παραπάνω m-script αρχείο, λαμβάνουμε στο τερματικό του Matlab την λύση του δεδομένου Π.Α.Τ. με την μέθοδο Runge – Kutta 4^{ης} τάξης (σε μορφή εκτύπωσης που καθορίζεται από την συνάρτηση `table`), τα αποτελέσματα της οποίας αναγράφονται στον Πίνακα V. Λαμβάνουμε επίσης την γραφική παράσταση της αναλυτικής λύσης, καθώς και της αριθμητικά υπολογιζόμενων με την μέθοδο σημείων, στους κόμβους της διαμέρισης (Βλ. Διάγραμμα V παρακάτω).

Επίλυση Π.Α.Τ. – Μέθοδος Runge-Kutta 4 ^{ης} Τάξης				
i	Τιμές t	Αναλυτική Λύση $y_{\text{analytical}}$	Μέθοδος RK4 y_{RK4}	Σφάλμα M. RK4 $\text{error}_{\text{RK4}}$
1	2.0	1.0000	1.0000	0.00000
2	2.1	1.1909	1.1909	2.7724e-07
3	2.2	1.3667	1.3667	3.9551e-07
4	2.3	1.5308	1.5308	4.3652e-07
5	2.4	1.6857	1.6857	4.3961e-07
6	2.5	1.8333	1.8333	4.244e-07
7	2.6	1.9750	1.9750	4.0095e-07
8	2.7	2.1118	2.1118	3.7446e-07
9	2.8	2.2444	2.2444	3.4763e-07
10	2.9	2.3737	2.3737	3.2179e-07
11	3.0	2.5000	2.5000	2.9758e-07

Πίνακας V.



Διάγραμμα V.

Σημειώνεται ότι τα δεδομένα του Πίνακα V αναγράφονται με 4 δεκαδικά ψηφία, όπως εμφανίζονται και στο Command Window, υπό την εντολή `format default`, παρόλο που η Matlab αποθηκεύει τις τιμές των αποτελεσμάτων σε περισσότερα δεκαδικά ψηφία (16 σημαντικά ψηφία). Η παρεχόμενη ακρίβεια της μεθόδου Runge – Kutta 4^{ης} τάξης (καθολικό σφάλμα $O(h^4)$), για την επιλεγόμενη τιμή βήματος $h = 0.1$ αποδίδει *προσεγγιστικά* ακρίβεια στο τέταρτο δεκαδικό ψηφίο, γεγονός που φαίνεται άμεσα και από τις τιμές του Πίνακα V, καθώς η επίλυση μέσω της μεθόδου ταυτίζεται με την αναλυτική λύση *το πολύ* στο τέταρτο δεκαδικό ψηφίο. Προς ανάδειξη του γεγονότος ότι η λύση της μεθόδου συμφωνεί με την αναλυτική λύση *το πολύ* στο τέταρτο δεκαδικό ψηφίο, μπορούμε να λάβουμε τις τιμές που προκύπτουν από το παραπάνω m-script «ανοίγοντας» τον δημιουργούμενο πίνακα διανυσμάτων, με την εντολή `open('T')`. Μέσω αυτής της εντολής, ο πίνακας που εμφανίζεται στο Command Window ανοίγει σε παράθυρο «κελιών» (μορφής τύπου Excel©), από το οποίο μπορεί να γίνει αντιγραφή των τιμών με όλη την ακρίβεια που προσφέρει το Matlab. Με αυτή την διαδικασία προκύπτει ο παρακάτω Πίνακας VI, που επί της ουσίας είναι μία «ακριβέστερη» εκδοχή του Πίνακα V παραπάνω.

Επίλυση Π.Α.Τ. – Μέθοδος Runge-Kutta 4^{ης} Τάξης (Βελτιωμένης Ακρίβειας)				
<i>i</i>	Τιμές <i>t</i>	Αναλυτική Λύση <i>y_{analytical}</i>	Μέθοδος RK4 <i>y_{RK4}</i>	Σφάλμα M. RK4 <i>error_{RK4}</i>
1	2.0	1	1	0
2	2.1	1.19090909090909	1.19090881366778	2.77241310575604e-07
3	2.2	1.36666666666667	1.36666627115693	3.95509738959632e-07
4	2.3	1.53076923076923	1.53076879424671	4.36522516800153e-07
5	2.4	1.68571428571429	1.68571384610724	4.39607046986978e-07
6	2.5	1.83333333333333	1.83333290893414	4.24399196008451e-07
7	2.6	1.97500000000000	1.97499959905083	4.00949173862131e-07
8	2.7	2.11176470588235	2.11176433141916	3.74463192454044e-07
9	2.8	2.24444444444444	2.24444409681679	3.47627658658922e-07
10	2.9	2.37368421052632	2.37368388873574	3.21790574187020e-07
11	3.0	2.50000000000000	2.49999970241977	2.97580231123362e-07

Πίνακας VI.

Παρατηρούμε ότι, αν και πράγματι σε ορισμένους κόμβους το αποτέλεσμα συμφωνεί με το αντίστοιχο της αναλυτικής λύσης *το πολύ* στο τέταρτο δεκαδικό ψηφίο, εντούτοις στην πλειοψηφία των περιπτώσεων, η επίλυση μέσω της μεθόδου Runge – Kutta 4^{ης} τάξης, συμφωνεί με την αναλυτική λύση ακόμα και στο *πέμπτο* δεκαδικό ψηφίο. Το γεγονός αυτό αναδεικνύει την υψηλή ακρίβεια της μεθόδου Runge – Kutta 4^{ης} τάξης, η οποία, παρά την σχετικά «μικρής ποιότητας» διαμέριση ($h = 0.1$), καταφέρνει να προσεγγίσει την λύση του ζητούμενου Π.Α.Τ. έως στο τέταρτο δεκαδικό ψηφίο.

6. Σύγκριση των αριθμητικών μεθόδων.

Έχοντας επιλύσει το Π.Α.Τ. με τις παραπάνω αριθμητικές μεθόδους, μπορούμε να προβούμε σε μεταξύ τους σύγκριση, σε ό,τι αφορά στην παρεχόμενη ακρίβεια για την λύση του. Δεδομένου ότι η απεικόνιση των τιμών λύσεως από κάθε μέθοδο σε κοινό διάγραμμα (σε μορφή αντίστοιχη των Διαγραμμάτων I – V παραπάνω) καταλήγει σε

μη – ευκρινή και «συνωστισμένη» γραφική παράσταση, γίνεται, αντί αυτού, απεικόνιση των απολύτων σφαλμάτων κάθε μεθόδου σε κοινό διάγραμμα (βλ. Διάγραμμα VI παρακάτω), όπου ο άξονας των σφαλμάτων τίθεται σε λογαριθμική κλίμακα (για ευκρινή ανάδειξη των σφαλμάτων, τα οποία διαφέρουν κατά πολλαπλές τάξεις μεγέθους). Επιπλέον, προς ανάδειξη της τάξης κάθε μεθόδου, πραγματοποιείται και γραφική απεικόνιση του καθολικού μεγίστου σφάλματος (global error) κάθε μεθόδου συναρτήσει του βήματος της θεωρούμενης διαμέρισης (βλ. Διάγραμμα VII παρακάτω). Και οι δύο άξονες του τελευταίου διαγράμματος τίθενται σε λογαριθμική κλίμακα προς ανάδειξη της εκάστοτε ασυμπτωτικής συμπεριφοράς του καθολικού σφάλματος, ώστε να καταστεί εμφανής η τάξη σύγκλισης της κάθε μεθόδου (ήτοι η σχέση $\text{error} \propto O(h^k)$, για την εκάστοτε τάξη k μεθόδου). Τα παραπάνω υλοποιούνται με το αρχείο τύπου m-script που παρατίθεται παρακάτω, και η επεξήγηση του οποίου ακολουθεί την γενική λογική των scripts των προηγούμενων εδαφίων.

MATLAB SCRIPT M-FILE (MethodComparisonEx2_1_Script.m):

```
clear; clc;
%      --- CFD Exercise 2.1 ---
% --- NUMERICAL METHODS COMPARISON ---
%
% (using EulerExplicitMethod.m,
%      EulerImplicitMethod.m,
%      RK2Method.m, RK3Method.m, RK4Method.m)
%
% -----
% --- DEFINITION OF PARAMETERS ---
% -----
% --- Definition of function handle ---
f = @(t,y) 1 + (t-y).^2;
% --- Definition of t_initial, t_final ---
t_initial = 2; t_final = 3;
% --- Definition of y0 ---
y0 = 1;
% --- Definition of partition step h ---
h = 0.1;
% --- Definition of accuracy epsilon ---
epsilon = 1e-5;
% --- Definition of number N of intervals ---
N = floor((t_final-t_initial)/h);
% --- Definition of node number vector ---
i = (1:(N+1))';
% --- Definition of time values vector ---
% (this vector will be overwritten below, but it is the same
%      for given h, t_initial and t_final)
t = (t_initial : h : t_final)';
% --- Definition of analytical solution function handle ---
y_analytical = @(t) t + (1-t).^-1;
% --- Definition of analytical solution at partition nodes ---
y_anal = y_analytical(t);
%
% -----
% --- SOLUTION OF I.V.P. ---
% --- (USING ALL METHODS) ---
% -----
% --- 1. Euler's Explicit method solution ---
```

```
[~,y_EE] = EulerExplicitMethod(f,t_initial,t_final,y0,h);
% --- Error Calculation ---
error_EE = abs(y_EE - y_anal);
% --- 2. Euler's Implicit method solution ---
[~,y_EI] = EulerImplicitMethod(f,t_initial,t_final,y0,h,epsilon);
% --- Error Calculation ---
error_EI = abs(y_EI - y_anal);
% --- 3. Runge-Kutta 2nd order method solution ---
[~,y_RK2] = RK2Method(f,t_initial,t_final,y0,h);
% --- Error Calculation ---
error_RK2 = abs(y_RK2 - y_anal);
% --- 4. Runge-Kutta 3rd order method solution ---
[~,y_RK3] = RK3Method(f,t_initial,t_final,y0,h);
% --- Error Calculation ---
error_RK3 = abs(y_RK3 - y_anal);
% --- 5. Runge-Kutta 4th order method solution ---
[~,y_RK4] = RK4Method(f,t_initial,t_final,y0,h);
% --- Error Calculation ---
error_RK4 = abs(y_RK4 - y_anal);
% -----
% --- Definition of table for data output ---
l = repmat(' ',N+1,1);
T1 = table(i,l, t,l, y_anal,l, y_EE,error_EE,l, y_EI,error_EI,l);
T2 = table(l, y_RK2,error_RK2,l, y_RK3,error_RK3,l, y_RK4,error_RK4);
T1 = renamevars(T1,"l"," "); T1 = renamevars(T1,"l_1"," ");
T1 = renamevars(T1,"l_2"," "); T1 = renamevars(T1,"l_3"," ");
T1 = renamevars(T1,"l_4"," ... "); T2 = renamevars(T2,"l"," ... ");
T2 = renamevars(T2,"l_1"," "); T2 = renamevars(T2,"l_2"," ");
disp(T1); disp(T2);
% -----
% --- ERROR COMPARISON PLOTS ---
% -----
% --- Plot solution error values vs. t values ---
figure;
semilogy(t, error_EE, 'r+', 'LineWidth', 0.5); hold on;
semilogy(t, error_EI, 'go', 'LineWidth', 0.5); hold on;
semilogy(t, error_RK2, 'bx', 'LineWidth', 0.5); hold on;
semilogy(t, error_RK3, 'm^', 'LineWidth', 0.5); hold on;
semilogy(t, error_RK4, 'kv', 'LineWidth', 0.5);
grid on;
xlabel('Time (t)');
ylabel('Absolute Error');
title('Numerical Method Comparison (h = 0.1)');
legend('Euler''s Explicit Method', ...
'Euler''s Implicit Method', 'R-K 2nd Order Method', ...
'R-K 3rd Order Method', 'R-K 4th Order Method', ...
'Position', [0.55,0.25,0.35,0.18]);
% -----
% --- Plot max abs error value vs. step size h ---
h = [0.1 0.05 0.025 0.01 0.005 0.0025 0.001];
max_err_EE = zeros(length(h),1);
max_err_EI = zeros(length(h),1);
max_err_RK2 = zeros(length(h),1);
max_err_RK3 = zeros(length(h),1);
max_err_RK4 = zeros(length(h),1);
for i = 1 : length(h)
    ti = (t_initial : h(i) : t_final)';
    y_anal = y_analytical(ti);
    [~,y_EE] = EulerExplicitMethod(f,t_initial,t_final,y0,h(i));
```

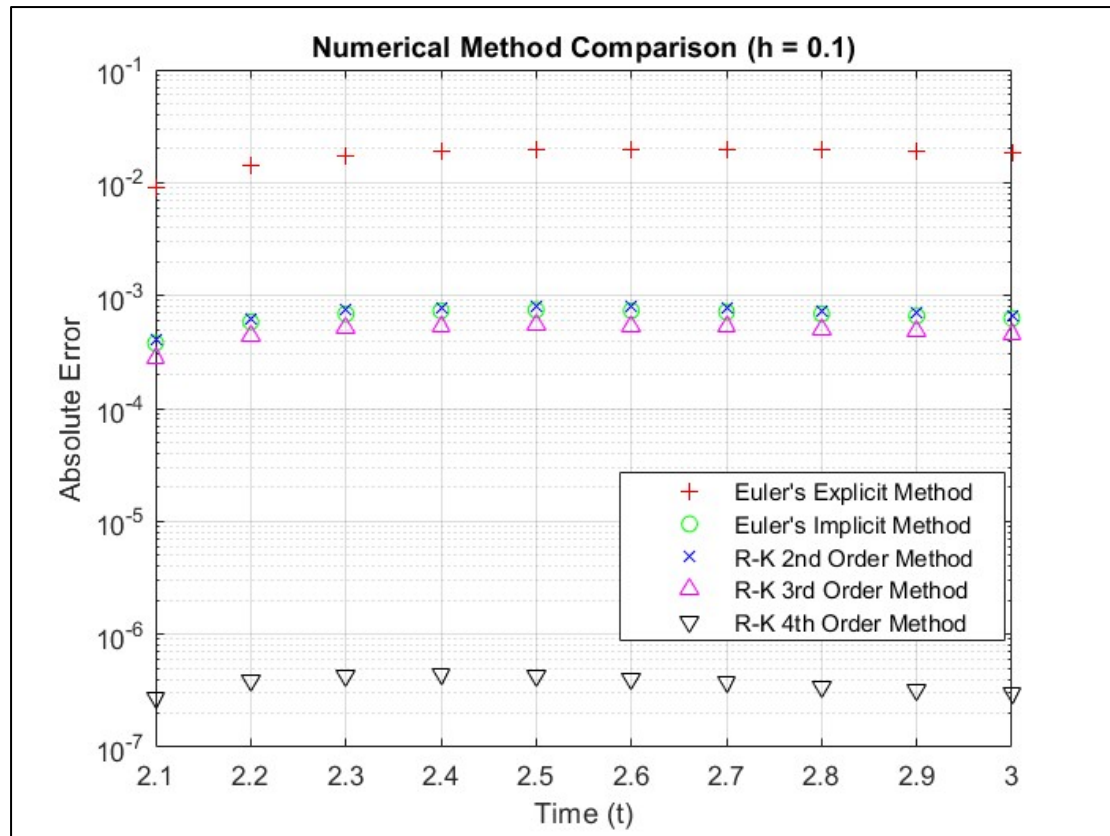
```

max_err_EE(i) = max(abs(y_EE - y_anal));
[~,y_EI] = EulerImplicitMethod(f,t_initial,t_final,y0,h(i),epsilon);
max_err_EI(i) = max(abs(y_EI - y_anal));
[~,y_RK2] = RK2Method(f,t_initial,t_final,y0,h(i));
max_err_RK2(i) = max(abs(y_RK2 - y_anal));
[~,y_RK3] = RK3Method(f,t_initial,t_final,y0,h(i));
max_err_RK3(i) = max(abs(y_RK3 - y_anal));
[~,y_RK4] = RK4Method(f,t_initial,t_final,y0,h(i));
max_err_RK4(i) = max(abs(y_RK4 - y_anal));
end
figure;
loglog(h, max_err_EE, 'r--', 'LineWidth', 0.5); hold on;
loglog(h, max_err_EI, 'g--', 'LineWidth', 0.5); hold on;
loglog(h, max_err_RK2, 'b--', 'LineWidth', 0.5); hold on;
loglog(h, max_err_RK3, 'm--', 'LineWidth', 0.5); hold on;
loglog(h, max_err_RK4, 'k--', 'LineWidth', 0.5);
grid on;
xlabel('Step Size (h)');
ylabel('Maximum Absolute Error');
title('Methods' Order of Convergence O(h^k)');
legend('Euler's Explicit Method', ...
       'Euler's Implicit Method', 'R-K 2nd Order Method', ...
       'R-K 3rd Order Method', 'R-K 4th Order Method', ...
       'Location', 'Best');
% End of script.

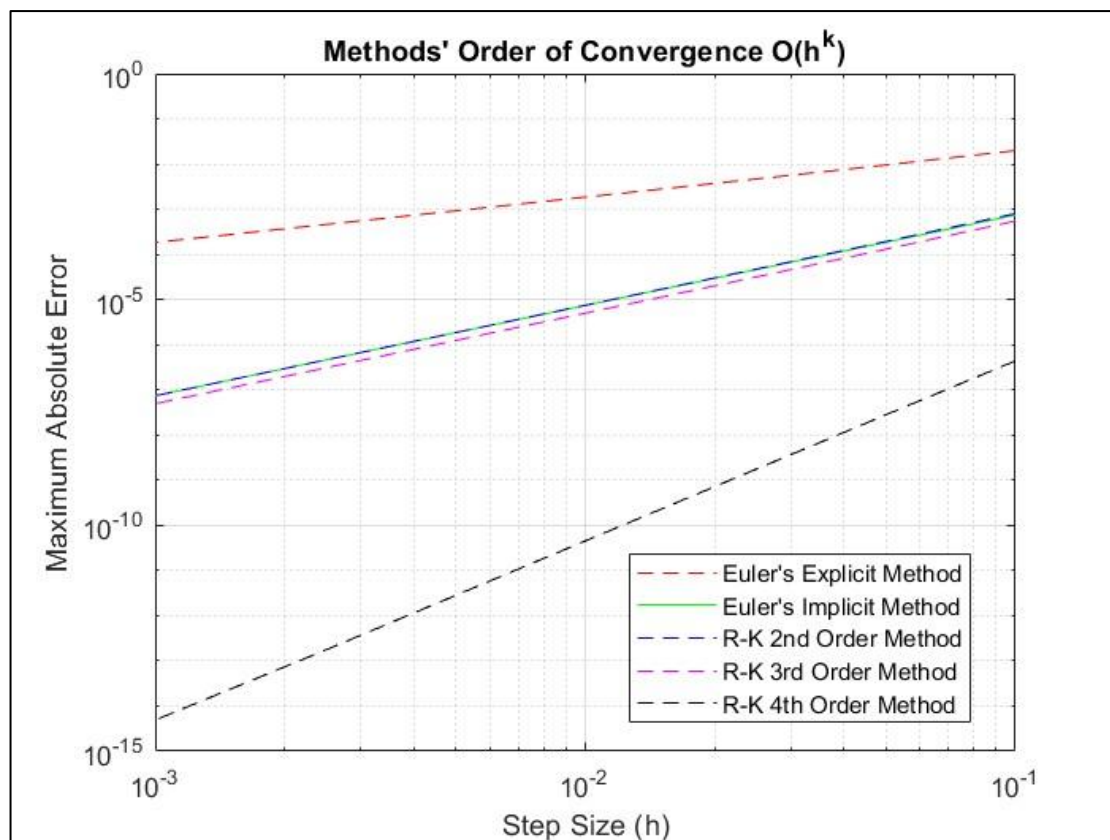
```

Command Window									
i	t	y_anal	y_EE	error_EE	y_EI	error_EI	...		
1	2	1	1	0	1	0			
2	2.1	1.1909	1.2	0.0090909	1.1913	0.00037879			
3	2.2	1.3667	1.381	0.014333	1.3672	0.00058278			
4	2.3	1.5308	1.5481	0.017307	1.5315	0.00068687			
5	2.4	1.6857	1.7046	0.018901	1.6864	0.0007327			
6	2.5	1.8333	1.853	0.019638	1.8341	0.00074419			
7	2.6	1.975	1.9948	0.019836	1.9757	0.00073547			
8	2.7	2.1118	2.1315	0.019693	2.1125	0.00071507			
9	2.8	2.2444	2.2638	0.019338	2.2451	0.00068822			
10	2.9	2.3737	2.3925	0.018851	2.3743	0.00065814			
11	3	2.5	2.5183	0.018287	2.5006	0.00062683			
...	y_RK2	error_RK2	y_RK3	error_RK3	y_RK4	error_RK4			
	1	0	1	0	1	0			
	1.1905	0.00040909	1.1906	0.00028167	1.1909	2.7724e-07			
	1.366	0.00062882	1.3662	0.00043217	1.3667	3.9551e-07			
	1.53	0.00074038	1.5303	0.00050809	1.5308	4.3652e-07			
	1.6849	0.00078896	1.6852	0.00054075	1.6857	4.3961e-07			
	1.8325	0.0008005	1.8328	0.0005481	1.8333	4.244e-07			
	1.9742	0.00079034	1.9745	0.00054066	1.975	4.0095e-07			
	2.111	0.00076769	2.1112	0.00052477	2.1118	3.7446e-07			
	2.2437	0.00073819	2.2439	0.00050427	2.2444	3.4763e-07			
	2.373	0.00070532	2.3732	0.00048154	2.3737	3.2179e-07			
	2.4993	0.00067122	2.4995	0.00045803	2.5	2.9758e-07			

Τρέχοντας το παραπάνω m-script αρχείο, λαμβάνουμε στο τερματικό του Matlab συγκεντρωτικά τα αποτελέσματα από την επίλυση του δεδομένου Π.Α.Τ. με καθεμία από τις ζητούμενες μεθόδους (σε μορφή εκτύπωσης που καθορίζεται από την συνάρτηση table, αλλά και με κατάλληλη χρήση της), όπως φαίνεται παραπάνω. Λαμβάνουμε επίσης και την γραφική απεικόνιση των απολύτων σφαλμάτων κάθε μίας



Διάγραμμα VI.



Διάγραμμα VII.

μεθόδου σε κοινό διάγραμμα (βλ. Διάγραμμα VI), όπου ο άξονας των σφαλμάτων τίθεται σε λογαριθμική κλίμακα, καθώς και το διάγραμμα του καθολικού μεγίστου σφάλματος (global error) κάθε μεθόδου συναρτήσει του βήματος της εκάστοτε επιλεγόμενης διαμέρισης (βλ. Διάγραμμα VII). Από το Διάγραμμα VI είναι προφανές ότι η Ρητή μέθοδος Euler παρουσιάζει το μεγαλύτερο απόλυτο σφάλμα σε κάθε βήμα, ενώ η μέθοδος Runge – Kutta 4^{ης} τάξης παρουσιάζει το μικρότερο απόλυτο σφάλμα σε κάθε βήμα, όπως άλλωστε αναμένεται, δεδομένου ότι η πρώτη μέθοδος είναι 1^{ης} τάξης ($O(h)$), ενώ η τελευταία 4^{ης} τάξης ($O(h^4)$), ως προς το μέγεθος του βήματος h . Για το δεδομένο βήμα $h = 0.1$, έπεται ότι η Ρητή μέθοδος Euler θα παρουσιάζει απόλυτο σφάλμα «προσεγγιστικά» περί την τάξη του 10^{-1} , ενώ η μέθοδος Runge – Kutta 4^{ης} τάξης περί την τάξη του 10^{-4} , όπως φαίνεται και στο Διάγραμμα VI παραπάνω.¹ Οι υπόλοιπες μέθοδοι παρουσιάζουν απόλυτα σφάλματα στην περιοχή του 10^{-3} , και, όπως αναμένεται, είναι ενδιάμεσης ακρίβειας από τις παραπάνω δύο «ακραίες» μεθόδους.

Από το Διάγραμμα VII, και συγκεκριμένα από τις κλίσεις των παραγόμενων «προσεγγιστικών» ευθειών θετικής κλίσης, γίνεται δυνατή η ανάδειξη της τάξης σύγκλισης κάθε μεθόδου.² Πράγματι, παρατηρώντας το Διάγραμμα VII, η κλίση της ευθείας της Ρητής μεθόδου Euler είναι πρακτικά 1, γεγονός που υποδεικνύει ότι η μέθοδος είναι 1^{ης} τάξης, ενώ για την Πεπλεγμένη μέθοδο Euler και την Runge – Kutta 2^{ης} τάξης οι ευθείες πρακτικά ταυτίζονται και η κλίση τους είναι περί το 2, γεγονός που υποδεικνύει 2^η τάξη σύγκλισης. Αντίστοιχα, για την μέθοδο Runge – Kutta 4^{ης} τάξης, η αντίστοιχη ευθεία έχει κλίση περί το 4, όπως δηλώνει και η τάξη της μεθόδου. Ωστόσο, παρατηρούμε ότι η ευθεία για την μέθοδο Runge – Kutta 3^{ης} τάξης είναι πρακτικά παράλληλη (!) προς την ευθείες που αντιστοιχούν στις μεθόδους 2^{ης} τάξης που αναφέρθηκαν παραπάνω. Το γεγονός αυτό είναι δηλωτικό του ότι ο χρησιμοποιούμενος αλγόριθμος (2.8) επί της ουσίας δεν περιγράφει μέθοδο 3^{ης} τάξης, αλλά 2^{ης} (!). Σύμφωνα με το βιβλίο του J. Butcher³, η μέθοδος Runge – Kutta 3^{ης} τάξης που μάλλον υπονοείται στις Σημειώσεις αντί της (2.8) είναι η εξής:

$$\left\{ \begin{array}{l} k_1 = f(t_i, y_i) \\ k_2 = f\left(x_i + \frac{h}{2}, y_i + \frac{h}{2}k_1\right) \\ k_3 = f(x_i + h, y_i - hk_1 + 2hk_2) \\ y_{i+1} = y_i + \frac{h}{6} \cdot (k_1 + 4k_2 + k_3) \end{array} \right\} \quad i = 1, 2, \dots, N, \quad y(1) \equiv y_0 \quad (2.8')$$

¹ Πιο αυστηρά, για μία μέθοδο τάξεως k , το απόλυτο σφάλμα σε κάθε σημείο της θεωρούμενης διαμέρισης, βήματος h , σε σχέση με την ακριβή τιμή, προσεγγίζεται από μία σχέση της μορφής:

$$|y_i - y(t_i)| \approx C(t_i) \cdot h^k$$

όπου η τιμή της C εξαρτάται από τους όρους τάξης $k + 1$ και έπειτα στο ανάπτυγμα Taylor της $y(t)$ στο σημείο t_i , και σε κάθε σημείο t_i της διαμέρισης μπορεί να παρουσιάζει σημαντική διακύμανση. Για το δεδομένο Π.Α.Τ., η διακύμανση αυτή δεν διαφέρει από την τιμή του h^k περισσότερο από μία ή δύο τάξεις μεγέθους, λόγω του ότι η λύση του Π.Α.Τ. είναι αρκετά «ομαλή».

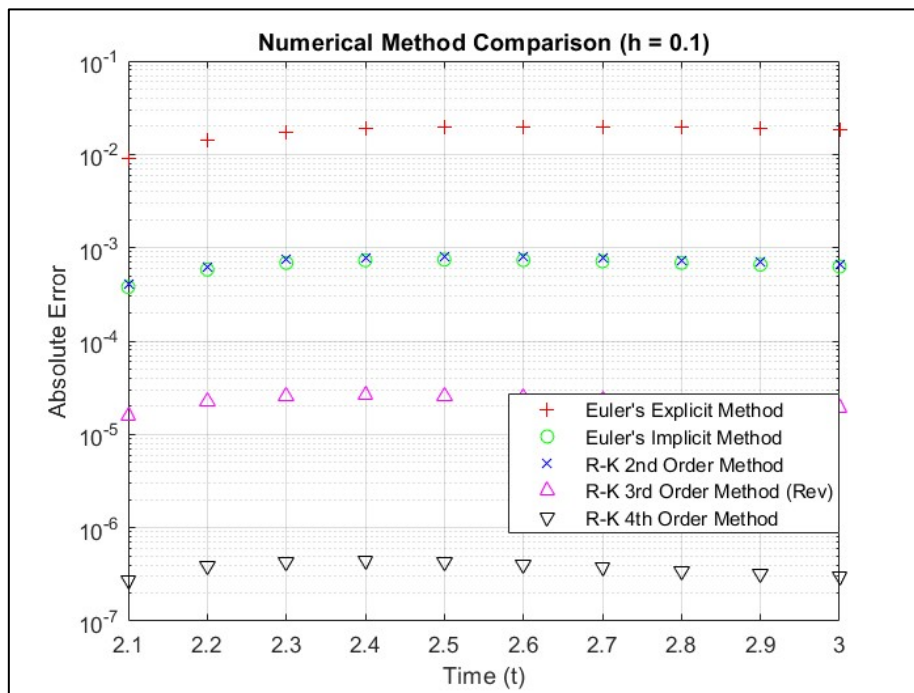
² Από την παραπάνω σχέση, λογαριθμίζοντας και τα δύο μέλη προκύπτει η σχέση:

$$\log|y_i - y(t_i)| \approx \log C(t_i) + k \cdot \log(h)$$

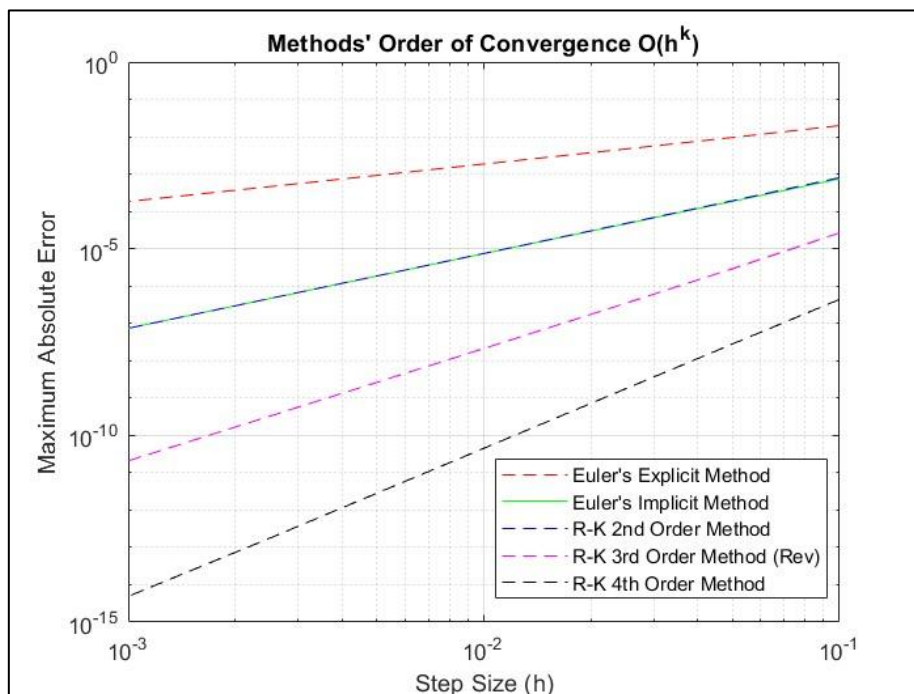
από όπου φαίνεται η «προσεγγιστικά» γραμμική εξάρτηση του λογαρίθμου του απολύτου σφάλματος σε σχέση με το $\log h$, με κλίση την τάξη k της μεθόδου.

³ Butcher, John C. (2003). *Numerical Methods for Ordinary Differential Equations*. John Wiley. ISBN 978-0-471-96758-3.

Κάνοντας την αντίστοιχη τροποποίηση στον κώδικα του αρχείου συνάρτησης τύπου m-function “RK3Method.m”, και τρέχοντας το m-script που παρουσιάστηκε στο παρόν εδάφιο, λαμβάνουμε τα παρακάτω Διαγράμματα VIII, IX, τα οποία είναι τα αντίστοιχα των VI, VII παραπάνω.



Διάγραμμα VIII.



Διάγραμμα IX.

Τα παραπάνω διαγράμματα δίνουν τα αναμενόμενα αποτελέσματα για την μέθοδο Runge – Kutta 3^{ης} τάξης, τόσο για τα σφάλματα σε κάθε κόμβο, όσο και για την τάξη της εν λόγω μεθόδου. Τα αποτελέσματα των άλλων μεθόδων μένουν ως παραπάνω. ■